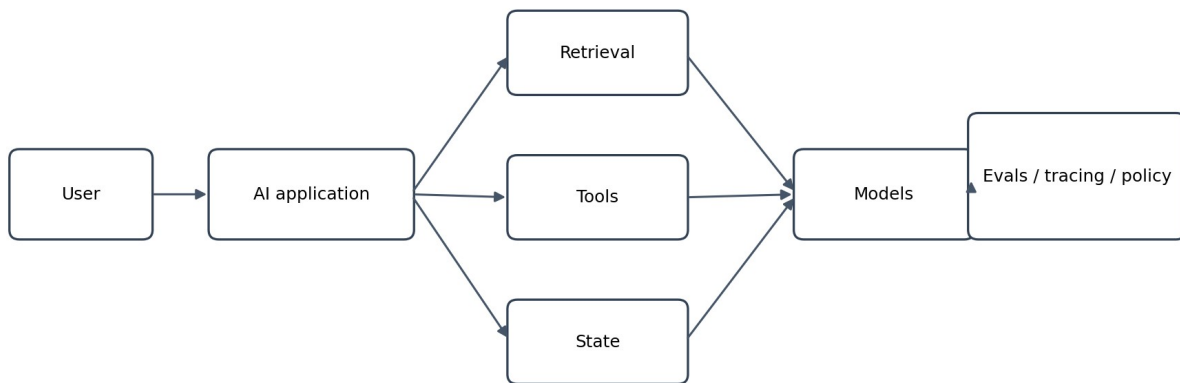


PRACTICAL AI ENGINEERING

From Tokens to Reliable Production Agents

The book in one picture



Anup Dangi

Engineering study edition - 2026

How to use this book

Read the chapter opening first, then study the diagrams, work through the numerical or concrete example, and finally build the lab. When revising, use the “Recall” block and redraw the architecture from memory before rereading.

Scope

This is an engineering handbook: it teaches how to build, measure and operate LLM applications and agents. It includes enough model internals to reason about inference, but it is not a pretraining or deep-learning-theory textbook.

Source policy

Each chapter links to primary documentation, original papers, or strong engineering sources. The book explains durable mental models; the linked sources provide deeper and version-sensitive detail.

Visual policy

Company architectures are redrawn as simplified educational diagrams rather than copied screenshots. Diagrams emphasize data flow, control flow, state ownership, failure boundaries and operational decisions.

STUDY RULE: For every topic, be able to answer five questions without notes: Why does it exist? How does it work? What fails? What trade-off did we choose? What would I build to test it?

Table of Contents

Part I

Foundations: models, tokens, inference, structured interfaces, prompts and routing

- 01. The AI Engineering Stack** - From model call to production system
- 02. Tokens, Decoding and Generation** - What happens after you send a prompt
- 03. Transformer Attention for Engineers** - Enough internals to reason about context and inference
- 04. Inference, KV Cache and Continuous Batching** - Why serving many users is a scheduling problem
- 05. Structured Outputs and Typed Interfaces** - Turn probabilistic text into dependable program inputs
- 06. Prompt Engineering as Software Engineering** - Prompts are versioned behavior, not magic incantations
- 07. Model Selection, Routing and Cascades** - Use the cheapest capable model, not one model for everything

Part II

Retrieval and RAG

- 08. Embeddings and Semantic Space** - Represent meaning as geometry
- 09. Vector Search: Exact, HNSW and IVF** - Trade recall, memory and latency deliberately
- 10. Chunking and Contextual Retrieval** - Preserve meaning when documents are broken apart
- 11. Hybrid Retrieval and Rank Fusion** - Combine lexical precision with semantic recall
- 12. Reranking and Context Assembly** - Retrieve broadly, then spend compute on precision
- 13. Production RAG Architecture** - Treat retrieval as a product subsystem

Part III

Agents and durable execution

- 14. Retrieval Evaluation** - Diagnose whether the model ever received the right evidence
- 15. Agent Fundamentals** - Workflows, agents and the execution loop
- 16. Tool Design and Agent-Computer Interfaces** - Good tools reduce reasoning burden
- 17. ReAct, Planning and Decomposition** - Reason enough to choose actions, not to create ceremony
- 18. Context Engineering** - Give the model the right information, not all information
- 19. Memory, State and Artifacts** - Do not use chat history as your database
- 20. Multi-Agent Systems** - Parallelism and specialization without coordination chaos

21. Durable Long-Running Agents - Hours of work require a harness, not one giant prompt

Part IV

Evals and observability

22. Evaluation Datasets and Failure Taxonomy - Turn “seems good” into measurable capability

23. LLM-as-Judge Calibration - Evaluate the evaluator

24. Agent Trajectory Evaluation - The final answer can be right for the wrong reasons

25. Observability and Tracing - Reconstruct what one AI run actually did

26. Cost Engineering - Measure cost per successful task

Part V

Production reliability, security and release

27. Latency Engineering - Optimize the critical path, not random components

28. SLOs and Resource Budgets for AI - Define healthy enough before incidents define it for you

29. Retries, Fallbacks and Graceful Degradation - Useful partial service beats uncontrolled collapse

30. Canary Releases and AI Regression Gates - Limit how many users discover your mistake

31. Security, Sandboxing and Agent Containment - Reduce blast radius with deterministic boundaries

32. Human Approval and Risk-Tiered Autonomy - Escalate by risk and irreversibility, not by fear

33. Verification, Provenance and Evidence-Bearing Outputs - Make correctness cheap to inspect

34. Continuous Improvement and Repair Loops - Measured self-improvement, not uncontrolled self-modification

35. Prompt Caching and Reuse - Reduce repeated prefix computation and cost

36. Model and Provider Failure Modes - Design for external AI dependency risk

37. Prompt Injection and Untrusted Context - Retrieved text is data, not authority

38. Background Jobs and Resumable Execution - Long work is a state machine

39. Production Checklists and Operational Readiness - A launch is a system property

40. Complete Production Agent Architecture - Putting the pieces together

Project Track

Project 1. Document Intelligence and RAG Engine - Build a source-aware technical-document assistant that can answer questions with inspectable evidence.

Project 2. Autonomous Coding Agent - Build a repository agent that can inspect code, edit files, run tests and make bounded progress across multiple sessions.

Project 3. Deep Research Agent - Build a long-running research system that discovers sources, assigns parallel investigations and produces evidence-bearing synthesis.

Project 4. AI Customer Support Platform - Build a production support assistant that routes simple questions, uses account tools safely and escalates uncertain cases to humans.

Project 5. AI SRE / Incident Response Agent - Build a read-heavy diagnostic agent that correlates metrics, logs, traces and deploys to produce evidence-backed incident hypotheses.

Appendices

- A. Equations worth remembering
- B. Production agent release checklist
- C. Source index
- D. Glossary and recall questions

PART I - FOUNDATIONS

Build the mental model, then test the failure modes.

01. The AI Engineering Stack

From model call to production system

Why this exists

An AI product is a system, not a model. The model is one probabilistic component inside an application that also owns context, tools, state, validation, user experience, observability, security and economics.

AI application as a system

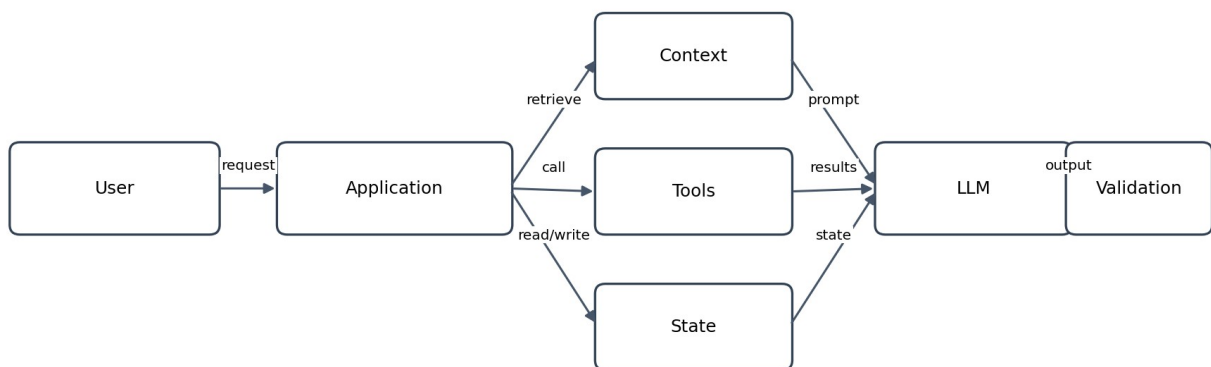


Figure: AI application as a system

Three concepts to retain

Probabilistic core: Model outputs can vary even when surrounding software is deterministic.

System objective: Optimize useful task success, not benchmark score in isolation.

Control plane: Budgets, routing, evaluation, permissions and deployment policy sit around the model.

The AI Engineering Stack - Mechanics

How it works

1. Separate model capability from product capability.
2. Make every external dependency explicit: retrieval, tools, storage, APIs.
3. Define what success means at the user-journey level.
4. Instrument cost, latency, quality and failures from the first prototype.
5. Keep deterministic operations in code when semantic reasoning is unnecessary.

Worked example

A support assistant receives a user question, retrieves account policy, calls an order API, validates the result, and returns a cited answer. The LLM is only the reasoning and language layer; identity, authorization, data access and side effects remain application responsibilities.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

The AI Engineering Stack - Failure Modes

Reason through these failures

- **Treating a demo prompt as the whole architecture.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Letting the model become the source of truth for business state.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Adding RAG, agents and memory before establishing a baseline.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Monitoring tokens but not user task success.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in the ai engineering stack is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

The AI Engineering Stack - Build and Recall

Lab

1. Build a minimal /answer endpoint around one model call.
2. Add structured output validation.
3. Add request-level latency and token accounting.
4. Write five failure cases before adding retrieval or tools.

Recall without notes

- Explain why the ai engineering stack exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Chip Huyen - Building LLM applications for production - <https://huyenchip.com/2023/04/11/llm-engineering.html>

Chip Huyen - Common pitfalls when building generative AI applications - <https://huyenchip.com/2025/01/16/ai-engineering-pitfalls.html>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

02. Tokens, Decoding and Generation

What happens after you send a prompt

Why this exists

Latency, cost, truncation and output variability all make more sense once you see generation as repeated next-token prediction rather than a single magical response.

Autoregressive generation

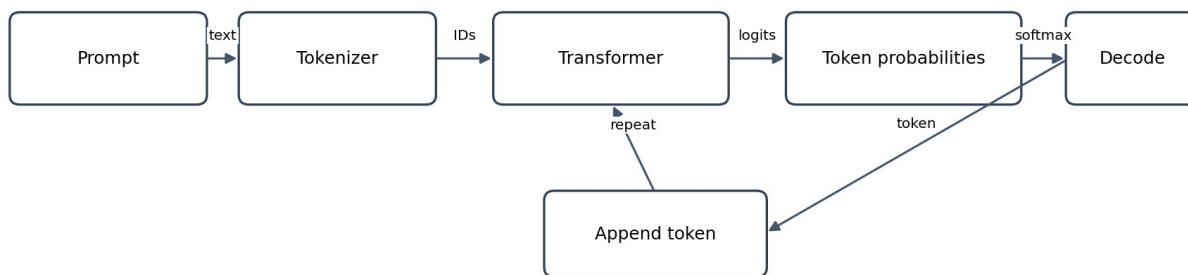


Figure: Autoregressive generation

Three concepts to retain

Tokenization: Text is converted into model-specific token IDs.

Next-token distribution: At each step the model scores possible next tokens.

Decoding policy: Temperature, top-p and stopping rules shape which token is chosen.

Tokens, Decoding and Generation - Mechanics

How it works

1. Tokenize the prompt.
2. Run a forward pass to produce logits.
3. Convert logits to a probability distribution.
4. Select or sample the next token.
5. Append it and repeat until a stop condition.

Worked example

If logits for three candidate tokens are [2.0, 1.0, 0.0], softmax converts them to approximately [0.665, 0.245, 0.090]. Lower temperature sharpens this distribution; higher temperature flattens it.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Tokens, Decoding and Generation - Failure Modes

Reason through these failures

- **Assuming characters equal tokens.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Generating verbose outputs when latency matters.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using high randomness for tasks requiring exact schemas.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Ignoring model-specific context limits.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in tokens, decoding and generation is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Tokens, Decoding and Generation - Build and Recall

Lab

1. Use a tokenizer to count tokens for the same paragraph across two models.
2. Generate 20 samples at multiple temperatures.
3. Measure output length and latency.
4. Create a deterministic structured-output task and tune decoding for reliability.

Recall without notes

- Explain why tokens, decoding and generation exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Hugging Face LLM Course - <https://huggingface.co/learn/llm-course/chapter1/8>

Chip Huyen - Generation configurations - <https://huyenchip.com/2024/01/16/sampling.html>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

03. Transformer Attention for Engineers

Enough internals to reason about context and inference

Why this exists

You do not need to train a frontier model to be an AI engineer, but attention, residual blocks and context dependence explain many practical constraints in retrieval, prompt design and inference serving.

Transformer block mental model

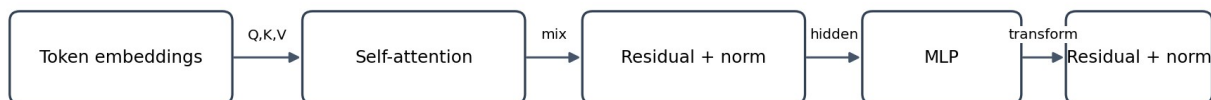


Figure: Transformer block mental model

Three concepts to retain

Embeddings: Tokens become vectors.

Attention: Each token mixes information from other tokens according to learned relevance.

Residual stack: Repeated attention and MLP blocks progressively transform representations.

Transformer Attention for Engineers - Mechanics

How it works

1. Project hidden states into Q, K and V.
2. Compute similarity scores QK^T .
3. Scale by $\sqrt{d_k}$.
4. Apply softmax to obtain attention weights.
5. Mix values and continue through residual blocks.

Worked example

For one query q and two keys k_1 and k_2 , scores $[2, 0]$ yield softmax weights about $[0.881, 0.119]$. The output is therefore dominated by v_1 but still incorporates v_2 .

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Transformer Attention for Engineers - Failure Modes

Reason through these failures

- **Treating attention weights as guaranteed explanations.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Assuming longer context is free.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Confusing training-time architecture with application-time prompting.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Ignoring position and recency effects in long inputs.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in transformer attention for engineers is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Transformer Attention for Engineers - Build and Recall

Lab

1. Implement scaled dot-product attention for a 3-token toy matrix in NumPy.
2. Print the attention matrix.
3. Change one key vector and observe redistribution.
4. Relate matrix size to sequence-length cost.

Recall without notes

- Explain why transformer attention for engineers exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Attention Is All You Need - <https://arxiv.org/abs/1706.03762>

The Annotated Transformer - <https://nlp.seas.harvard.edu/annotated-transformer/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

04. Inference, KV Cache and Continuous Batching

Why serving many users is a scheduling problem

Why this exists

A model that is fast for one request can perform poorly under concurrency. Production serving is dominated by memory, KV-cache allocation, batching and scheduler behavior as much as raw FLOPS.

LLM serving under concurrency

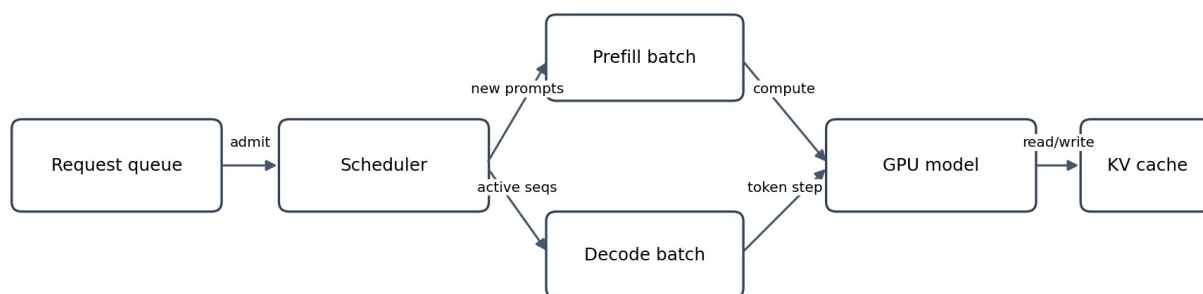


Figure: LLM serving under concurrency

Three concepts to retain

Prefill: Process the prompt and populate attention cache.

Decode: Generate one or a few tokens iteratively.

KV cache: Store past key/value tensors so earlier tokens are not recomputed.

Continuous batching: Insert and remove requests during decoding instead of waiting for a static batch.

Inference, KV Cache and Continuous Batching - Mechanics

How it works

1. Estimate model-weight memory.
2. Budget remaining GPU memory for KV cache.
3. Admit sequences according to token footprint.
4. Batch decode steps across active requests.
5. Remove finished requests and admit new ones.

Worked example

For an illustrative 32-layer model with 8 KV heads, head dimension 128 and FP16 cache, KV cache is about 128 KB per token: $2 \times 32 \times 8 \times 128 \times 2$ bytes. An 8k-token sequence therefore approaches 1 GB of KV state.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Inference, KV Cache and Continuous Batching - Failure Modes

Reason through these failures

- **Sizing concurrency only by requests, not tokens.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Allowing one very long sequence to consume cache headroom.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Static batching that idles while short requests finish.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Optimizing throughput while violating interactive latency.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in inference, kv cache and continuous batching is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Inference, KV Cache and Continuous Batching - Build and Recall

Lab

1. Simulate requests with different prompt/output lengths.
2. Compare static batching with continuous admission.
3. Track active tokens and completed requests over time.
4. Plot throughput versus p95 latency.

Recall without notes

- Explain why inference, kv cache and continuous batching exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

vLLM project - <https://github.com/vllm-project/vllm>

vLLM paper - PagedAttention - <https://arxiv.org/abs/2309.06180>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

05. Structured Outputs and Typed Interfaces

Turn probabilistic text into dependable program inputs

Why this exists

Applications become easier to test and compose when the model produces a constrained contract that can be validated before downstream code trusts it.

Typed LLM boundary

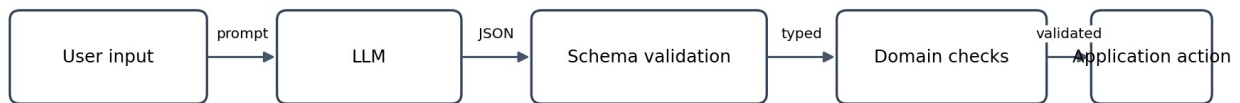


Figure: Typed LLM boundary

Three concepts to retain

Schema: Explicit fields, types, enums and required properties.

Validation: Reject or repair outputs that violate the contract.

Semantic correctness: Schema-valid data can still be factually wrong; type safety is necessary but not sufficient.

Structured Outputs and Typed Interfaces - Mechanics

How it works

1. Define the contract before the prompt.
2. Use constrained/structured output support when available.
3. Parse and validate at the boundary.
4. Distinguish parse failure from semantic failure.
5. Keep schemas small enough for reliable generation.

Worked example

An invoice extractor should return customer, currency, subtotal, tax and total as typed fields. A total of -900 can be syntactically valid yet semantically invalid; add domain validation such as non-negative totals and arithmetic checks.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Structured Outputs and Typed Interfaces - Failure Modes

Reason through these failures

- **Parsing free-form prose with brittle regex.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Assuming schema validation proves truth.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using enormous deeply nested schemas.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Retrying unchanged requests indefinitely after deterministic validation errors.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in structured outputs and typed interfaces is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Structured Outputs and Typed Interfaces - Build and Recall

Lab

1. Create a Pydantic/JSON schema for a support ticket.
2. Generate 100 outputs.
3. Measure parse rate and semantic-rule failures.
4. Add a repair path only for recoverable violations.

Recall without notes

- Explain why structured outputs and typed interfaces exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenAI - Structured Outputs - <https://developers.openai.com/api/docs/guides/structured-outputs>

Pydantic documentation - <https://docs.pydantic.dev/latest/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

06. Prompt Engineering as Software Engineering

Prompts are versioned behavior, not magic incantations

Why this exists

Prompting is the cheapest behavior layer to change, but unstructured prompt tweaking becomes fragile without examples, evaluation and version control.

Prompt construction

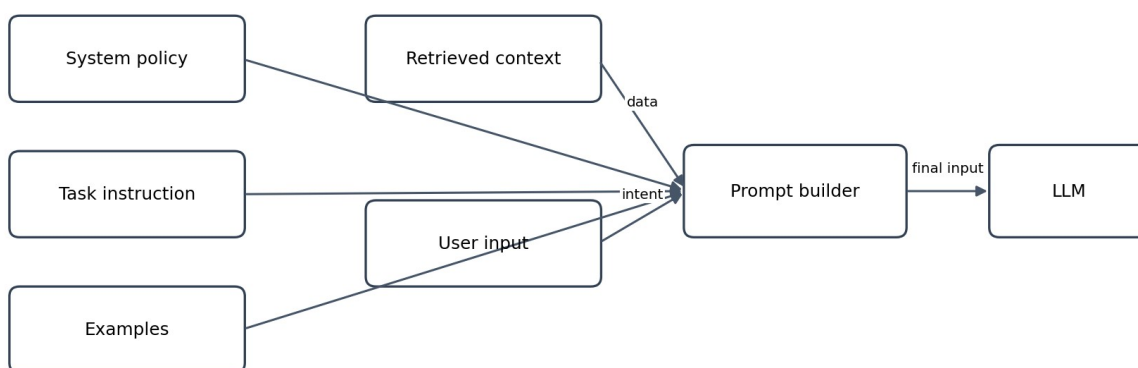


Figure: Prompt construction

Three concepts to retain

Instruction hierarchy: Separate system policy, developer logic, user intent and retrieved data.

Few-shot examples: Demonstrations can specify format and decision boundaries.

Prompt regression: A wording change can improve one slice while harming another.

Prompt Engineering as Software Engineering - Mechanics

How it works

1. State objective and constraints clearly.
2. Provide examples for ambiguous decisions.
3. Separate data from instructions.
4. Version prompts with the application.
5. Use evals to promote changes.

Worked example

A classification prompt improves from 87% to 93% overall after adding examples, but the refund-request slice drops from 96% to 80%. The prompt should not ship until slice-level regressions are understood.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Prompt Engineering as Software Engineering - Failure Modes

Reason through these failures

- **Endless prompt tweaking without an eval set.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Putting untrusted retrieved text in the instruction layer.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using chain-of-thought exposure as a debugging strategy.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Encoding business rules only in natural language when code can enforce them.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in prompt engineering as software engineering is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Prompt Engineering as Software Engineering - Build and Recall

Lab

1. Create a 40-example classification eval set.
2. Test zero-shot, few-shot and structured-output prompts.
3. Measure slice accuracy.
4. Version the winning prompt and record rationale.

Recall without notes

- Explain why prompt engineering as software engineering exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Lilian Weng - Prompt Engineering - <https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/>

Anthropic prompt engineering docs - <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

07. Model Selection, Routing and Cascades

Use the cheapest capable model, not one model for everything

Why this exists

Production workloads contain a distribution of task difficulty. Routing allows the system to reserve expensive reasoning for requests that need it.

Model routing

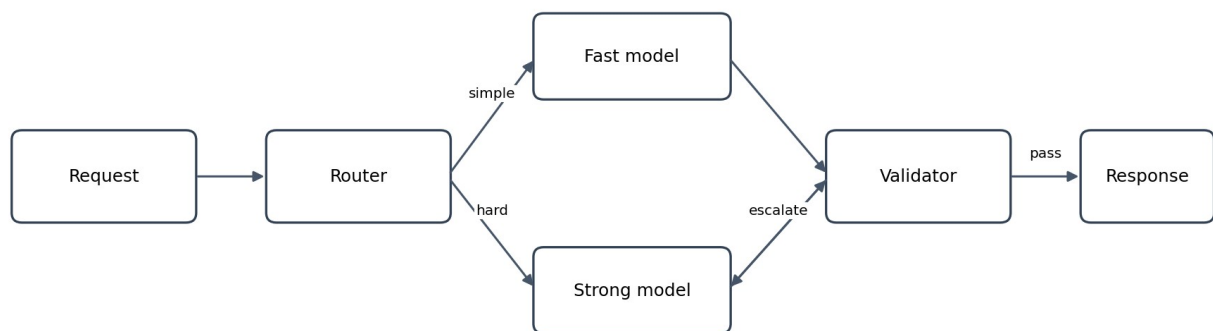


Figure: Model routing

Three concepts to retain

Capability fit: Choose models by application evals, not broad benchmark rank.

Router: Predict the cheapest model likely to satisfy the task.

Escalation: Fallback to a stronger model when confidence or validation fails.

Model Selection, Routing and Cascades - Mechanics

How it works

1. Define task slices.
2. Benchmark candidate models on each slice.
3. Estimate cost and latency per model.
4. Design routing signals.
5. Evaluate end-to-end utility, including routing mistakes.

Worked example

If 70% of support questions are simple FAQs, a smaller model can handle those while a stronger model handles policy ambiguity and tool-heavy cases. Savings come only if router errors do not create expensive retries or quality failures.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Model Selection, Routing and Cascades - Failure Modes

Reason through these failures

- **Routing based solely on prompt length.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Ignoring model differences in tool/JSON reliability.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Fallback loops between providers.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Optimizing cost while increasing verification burden.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in model selection, routing and cascades is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Model Selection, Routing and Cascades - Build and Recall

Lab

1. Build two-model routing using a simple classifier.
2. Compare always-strong, always-cheap and routed strategies.
3. Measure task success, p95 latency and cost/success.
4. Add escalation after failed validation.

Recall without notes

- Explain why model selection, routing and cascades exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Chip Huyen - Predictive Human Preference: From Model Ranking to Model Routing - <https://huyenchip.com/2024/02/28/predictive-human-preference.html>

OpenAI model guidance - <https://developers.openai.com/api/docs/guides/latest-model>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

PART II - RETRIEVAL AND RAG

Build the mental model, then test the failure modes.

08. Embeddings and Semantic Space

Represent meaning as geometry

Why this exists

Embeddings make semantic retrieval, clustering and similarity possible by converting text or other objects into vectors whose geometry reflects useful relationships.

Embedding retrieval

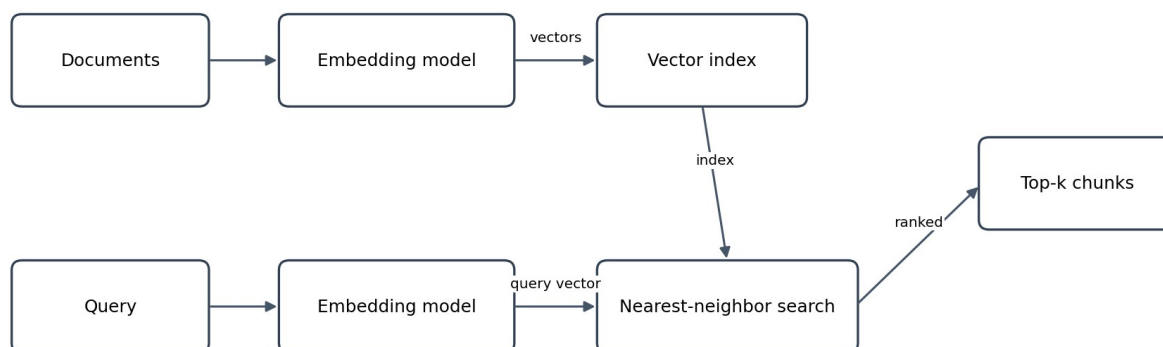


Figure: Embedding retrieval

Three concepts to retain

Vector representation: A fixed-length numeric representation of an input.

Distance metric: Cosine, inner product or Euclidean distance define similarity.

Task fit: Embedding quality depends on domain, language and retrieval objective.

Embeddings and Semantic Space - Mechanics

How it works

1. Embed documents and queries with the same compatible model.
2. Normalize when the chosen metric assumes it.
3. Store vectors alongside stable document IDs and metadata.
4. Search nearest neighbors.
5. Evaluate retrieval on domain examples.

Worked example

For vectors $a=[1,1]$ and $b=[2,0]$, cosine similarity is $2/(\sqrt{2} \cdot 2)=0.707$. The value measures angle, not raw magnitude.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
query_vec = embed(query)
rows = db.search(query_vec, k=10)
# evaluate relevance before generation
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Embeddings and Semantic Space - Failure Modes

Reason through these failures

- **Assuming semantic similarity equals factual relevance.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Mixing embedding models without reindexing.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Ignoring exact identifiers and rare terms.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Evaluating only with anecdotal queries.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in embeddings and semantic space is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Embeddings and Semantic Space - Build and Recall

Lab

1. Create embeddings for 100 technical paragraphs.
2. Compute pairwise cosine similarity.
3. Inspect nearest neighbors.
4. Add adversarial exact-ID queries to expose semantic-search limits.

Recall without notes

- Explain why embeddings and semantic space exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenAI embeddings guide - <https://developers.openai.com/api/docs/guides/embeddings>

Sentence Transformers documentation - <https://www.sbert.net/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

09. Vector Search: Exact, HNSW and IVF

Trade recall, memory and latency deliberately

Why this exists

Nearest-neighbor retrieval at scale is an indexing problem. Approximate indexes are powerful because they sacrifice a controlled amount of recall to avoid scanning every vector.

Exact vs approximate nearest-neighbor search

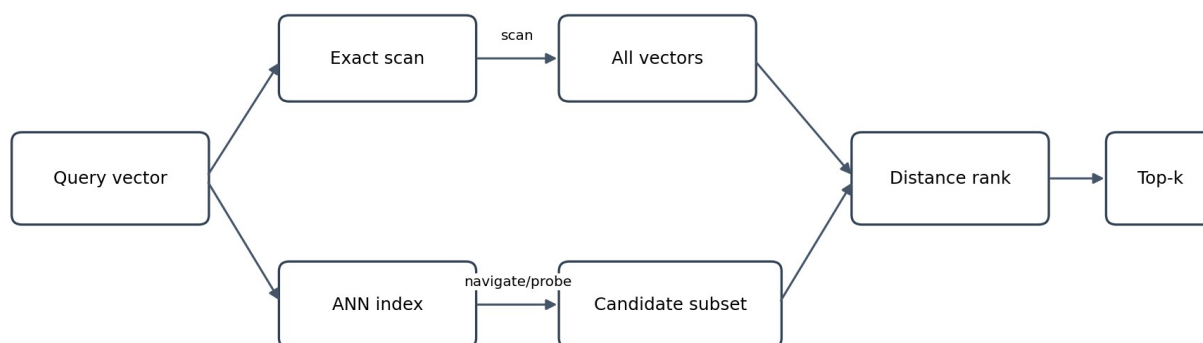


Figure: Exact vs approximate nearest-neighbor search

Three concepts to retain

Exact search: Scan all vectors; maximum recall and simple baseline.

HNSW: Navigate a multilayer proximity graph.

IVF: Partition vector space into lists and probe a subset.

Vector Search: Exact, HNSW and IVF - Mechanics

How it works

1. Start with exact search as ground truth.
2. Measure recall@k against exact results.
3. Tune HNSW search breadth or IVF probes.
4. Track memory and build cost.
5. Re-evaluate after adding metadata filters.

Worked example

If exact top-10 contains 10 relevant neighbors and ANN returns 9 of them, recall@10 relative to exact is 0.9. Lowering search breadth may reduce latency but can lower that recall.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
exact = IndexFlat(vectors)
approx = HNSW(vectors)
recall = overlap(exact.topk(q), approx.topk(q)) / k
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Vector Search: Exact, HNSW and IVF - Failure Modes

Reason through these failures

- **Adding ANN before measuring exact baseline.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using index defaults blindly.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Filters applied after ANN returning too few results.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Ignoring reindex/build time during deployment.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in vector search: exact, hnsw and ivf is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Vector Search: Exact, HNSW and IVF - Build and Recall

Lab

1. Index 100k vectors with FAISS Flat, HNSW and IVF.
2. Measure build time, query p95, memory and recall@10.
3. Sweep one tuning parameter.
4. Plot latency versus recall.

Recall without notes

- Explain why vector search: exact, hns w and ivf exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

FAISS wiki - <https://github.com/facebookresearch/faiss/wiki/>

FAISS - Guidelines to choose an index - <https://github.com/facebookresearch/faiss/wiki/Guidelines-to-choose-an-index>

pgvector - <https://github.com/pgvector/pgvector>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

10. Chunking and Contextual Retrieval

Preserve meaning when documents are broken apart

Why this exists

Retrieval systems often fail not because the embedding is weak but because the indexed chunk has lost the context needed to identify what it means.

Context-aware chunking

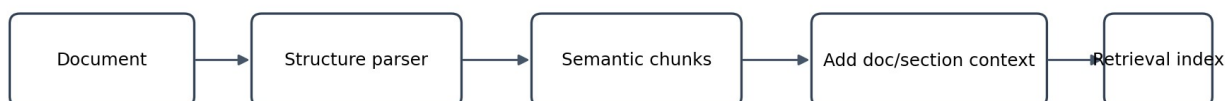


Figure: Context-aware chunking

Three concepts to retain

Chunk boundary: A retrieval unit should be coherent enough to stand on its own.

Contextualization: Prepend or encode document-level context with the chunk.

Hierarchy: Parent/child retrieval can combine precise matching with broader surrounding context.

Chunking and Contextual Retrieval - Mechanics

How it works

1. Parse document structure before blind splitting.
2. Respect headings, tables and semantic boundaries.
3. Attach metadata such as document, section and date.
4. Contextualize ambiguous chunks.
5. Evaluate multiple chunking strategies on actual questions.

Worked example

The chunk “Revenue grew 3% over the prior quarter” is difficult to retrieve for “ACME Q2 2023 growth” unless company and quarter are preserved in metadata or contextual text.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Chunking and Contextual Retrieval - Failure Modes

Reason through these failures

- **Fixed 500-token splitting across headings/tables.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Chunks too small to answer; chunks too large to retrieve precisely.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Discarding page/section provenance.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Treating OCR noise as clean text.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in chunking and contextual retrieval is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Chunking and Contextual Retrieval - Build and Recall

Lab

1. Take one long report.
2. Create fixed, heading-aware and contextualized chunks.
3. Build the same retrieval index for each.
4. Compare recall on 20 questions.

Recall without notes

- Explain why chunking and contextual retrieval exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - Contextual Retrieval - <https://www.anthropic.com/engineering/contextual-retrieval>

LangChain text splitters concepts - https://python.langchain.com/docs/concepts/text_splitters/

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

11. Hybrid Retrieval and Rank Fusion

Combine lexical precision with semantic recall

Why this exists

Dense retrieval is strong at meaning but can miss exact names, identifiers and rare tokens. Lexical methods provide a complementary signal.

Hybrid retrieval

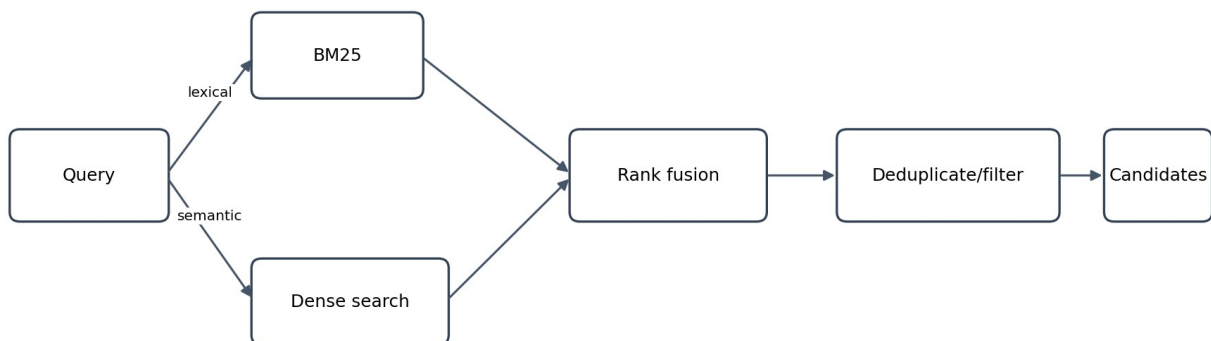


Figure: Hybrid retrieval

Three concepts to retain

BM25: Sparse lexical scoring based on term frequency and rarity.

Dense retrieval: Embedding similarity for semantic matching.

Fusion: Combine ranked lists instead of trusting one retriever.

Hybrid Retrieval and Rank Fusion - Mechanics

How it works

1. Run lexical and dense searches independently.
2. Normalize or rank-fuse results.
3. Deduplicate documents/chunks.
4. Apply metadata constraints.
5. Pass candidates to a reranker when needed.

Worked example

The query “ERR_AUTH_1042” should heavily reward an exact lexical match, while “why does login fail after token expiry?” may benefit from semantic retrieval. Hybrid search can recover both.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Hybrid Retrieval and Rank Fusion - Failure Modes

Reason through these failures

- **Averaging incomparable raw scores.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Letting dense retrieval drown exact identifiers.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Duplicate chunks consuming top-k.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Applying filters in a way that silently harms ANN recall.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in hybrid retrieval and rank fusion is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Hybrid Retrieval and Rank Fusion - Build and Recall

Lab

1. Implement BM25 with a small corpus.
2. Combine it with embedding retrieval using reciprocal-rank fusion.
3. Create queries with exact IDs and semantic paraphrases.
4. Compare recall against each retriever alone.

Recall without notes

- Explain why hybrid retrieval and rank fusion exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - Contextual Retrieval - <https://www.anthropic.com/engineering/contextual-retrieval>

Elasticsearch BM25 similarity - <https://www.elastic.co/guide/en/elasticsearch/reference/current/index-modules-similarity.html>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

12. Reranking and Context Assembly

Retrieve broadly, then spend compute on precision

Why this exists

The first retriever should usually optimize recall. A more expensive reranker can then score a smaller candidate set with richer interaction between query and document.

Two-stage retrieval

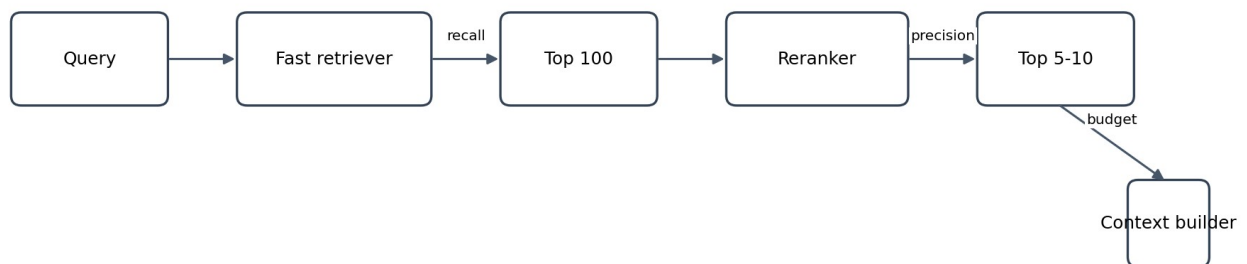


Figure: Two-stage retrieval

Three concepts to retain

Two-stage retrieval: Cheap broad retrieval followed by expensive precise scoring.

Cross-encoder style scoring: Jointly evaluate query and candidate.

Context budget: Select evidence that maximizes usefulness under token limits.

Reranking and Context Assembly - Mechanics

How it works

1. Retrieve 50-200 candidates cheaply.
2. Rerank the candidate pool.
3. Remove near-duplicates.
4. Prefer evidence diversity when useful.
5. Assemble context with provenance and budget limits.

Worked example

Retrieving 100 candidates at 20 ms and reranking only those can be far cheaper than applying a heavy relevance model over millions of chunks while still improving precision at top-5.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Reranking and Context Assembly - Failure Modes

Reason through these failures

- **Reranking too few candidates after poor recall.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Treating reranker score as factual correctness.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Stuffing all high-scoring chunks into context.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Losing citation mapping during compaction.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in reranking and context assembly is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Reranking and Context Assembly - Build and Recall

Lab

1. Build a retrieve-50 then rerank-5 pipeline.
2. Compare answer quality against retrieve-5 only.
3. Measure added latency and token cost.
4. Inspect cases where reranking harms diversity.

Recall without notes

- Explain why reranking and context assembly exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Sentence Transformers - Cross Encoders -

https://www.sbert.net/examples/cross_encoder/applications/README.html

Eugene Yan - LLM patterns - <https://eugeneyan.com/writing/llm-patterns/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

13. Production RAG Architecture

Treat retrieval as a product subsystem

Why this exists

A production RAG system must own ingestion, access control, freshness, retrieval quality, citations and failure recovery. “Vector DB + prompt” is only the happy-path core.

Production RAG - ingestion and query planes

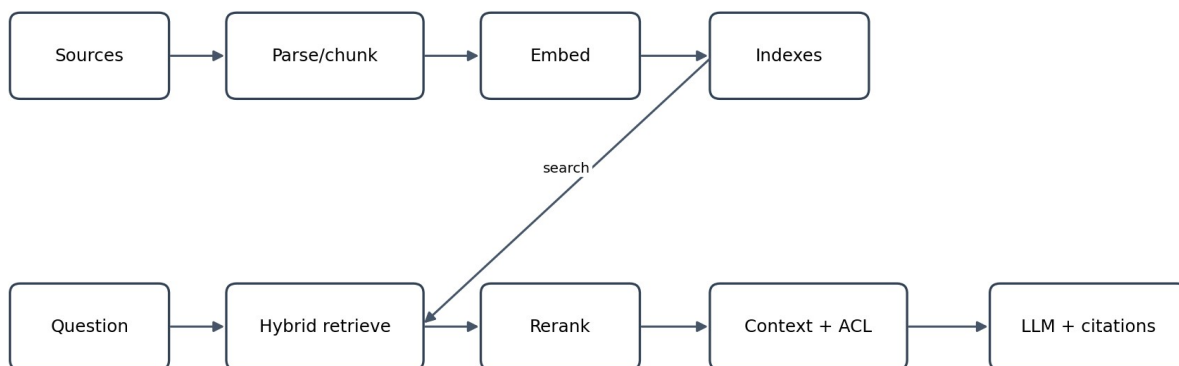


Figure: Production RAG - ingestion and query planes

Three concepts to retain

Ingestion plane: Parse, normalize, chunk, embed and index documents.

Query plane: Rewrite, retrieve, rerank and assemble context.

Governance: Permissions and freshness must survive every retrieval step.

Production RAG Architecture - Mechanics

How it works

1. Version documents and embeddings.
2. Keep source IDs/page locations with each chunk.
3. Filter by authorization before evidence reaches the model.
4. Evaluate retrieval independently of generation.
5. Support reindex/backfill and stale-index detection.

Worked example

If a policy document changes today but the vector index updates tomorrow, the model can confidently cite obsolete policy. Freshness is therefore a correctness contract, not an indexing detail.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Production RAG Architecture - Failure Modes

Reason through these failures

- **Permission leakage through shared indexes.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Stale embeddings after source edits.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Silent partial ingestion.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Hallucinated citations that do not support claims.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in production rag architecture is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Production RAG Architecture - Build and Recall

Lab

1. Build ingestion status tracking: discovered, parsed, chunked, embedded, indexed.
2. Attach source/page metadata.
3. Add a permission filter.
4. Create a retrieval audit log for every answer.

Recall without notes

- Explain why production rag architecture exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - Contextual Retrieval - <https://www.anthropic.com/engineering/contextual-retrieval>

Eugene Yan - LLM patterns - <https://eugeneyan.com/writing/llm-patterns/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

PART III - AGENTS

Build the mental model, then test the failure modes.

14. Retrieval Evaluation

Diagnose whether the model ever received the right evidence

Why this exists

Generation quality is downstream of retrieval. If the correct evidence never enters context, prompt engineering cannot reliably recover the answer.

Evaluate retrieval separately

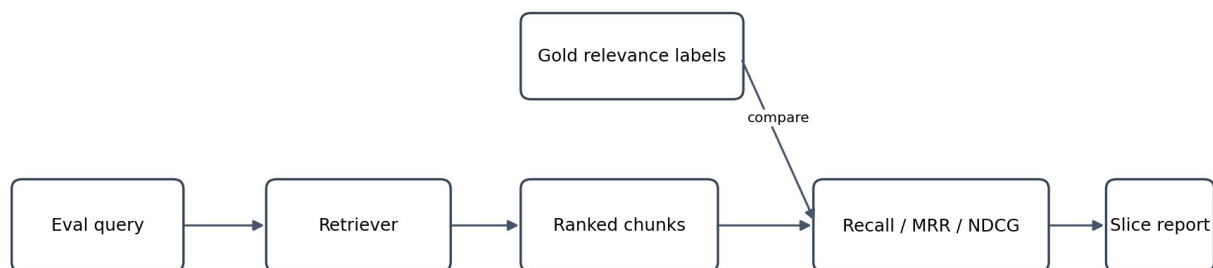


Figure: Evaluate retrieval separately

Three concepts to retain

Recall@k: How much of the relevant evidence appears in the top-k.

MRR: Rewards ranking the first relevant result early.

NDCG: Handles graded relevance and ranking position.

Retrieval Evaluation - Mechanics

How it works

1. Create query-evidence relevance labels.
2. Evaluate retriever before the LLM.
3. Slice by query type and document class.
4. Compare exact versus ANN retrieval.
5. Track changes whenever chunking/index/model changes.

Worked example

For one query with two relevant chunks, if top-5 contains one of them, recall@5 is 0.5. An answer can still look fluent, but the retrieval system has already lost half the required evidence.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Retrieval Evaluation - Failure Modes

Reason through these failures

- **Evaluating only final answer quality.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using synthetic questions that are easier than users.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Changing embedding model without rebuilding baselines.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Ignoring low-frequency failure slices.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in retrieval evaluation is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Retrieval Evaluation - Build and Recall

Lab

1. Create 50 questions with labeled supporting chunks.
2. Compute recall@5 and MRR for lexical, dense and hybrid search.
3. Inspect the 10 worst queries.
4. Change chunking and rerun.

Recall without notes

- Explain why retrieval evaluation exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

BEIR benchmark paper - <https://arxiv.org/abs/2104.08663>

Information retrieval evaluation overview - Stanford IR book -

<https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-of-ranked-retrieval-results-1.html>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

15. Agent Fundamentals

Workflows, agents and the execution loop

Why this exists

An agent is useful when the system must decide what action to take next based on intermediate results. Many tasks remain better as deterministic workflows.

The agent loop

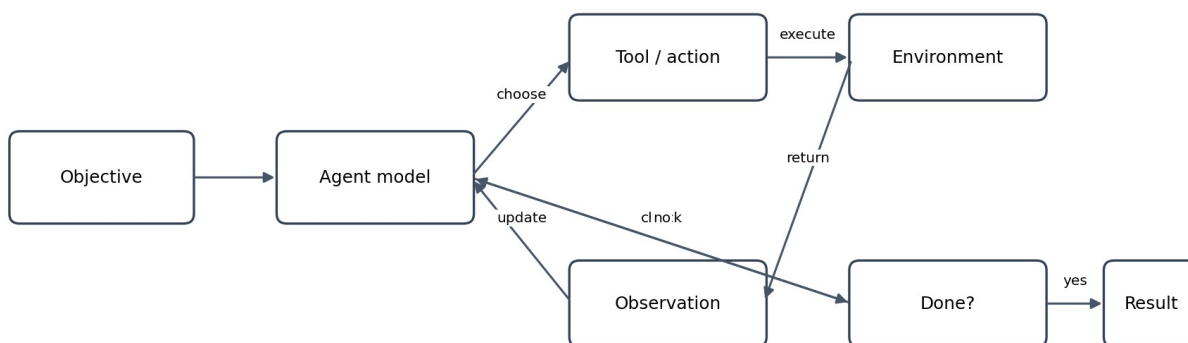


Figure: The agent loop

Three concepts to retain

Workflow: Predetermined control flow.

Agent: Model dynamically selects actions and tools.

Environment: Tools and state create the world in which the agent acts.

Agent Fundamentals - Mechanics

How it works

1. Receive objective and state.
2. Reason about next useful action.
3. Call a tool or model.
4. Observe result.
5. Stop when success/termination conditions are met.

Worked example

Refund processing with fixed eligibility rules should be a workflow. Investigating a complex production incident may justify an agent because the next query depends on what previous logs reveal.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
while steps < MAX_STEPS:
    action = model.next_action(state)
    observation = tools.execute(action)
    state = update(state, observation)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Agent Fundamentals - Failure Modes

Reason through these failures

- **Calling every multi-step prompt an agent.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using agents for deterministic transformations.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No termination condition.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Hiding tool behavior behind opaque frameworks.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in agent fundamentals is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Agent Fundamentals - Build and Recall

Lab

1. Implement a simple loop with two tools: search_docs and calculator.
2. Log each action/observation.
3. Add MAX_STEPS=6.
4. Compare with a fixed two-step workflow on the same task set.

Recall without notes

- Explain why agent fundamentals exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - Building effective agents - <https://www.anthropic.com/engineering/building-effective-agents>

OpenAI Agents SDK guide - <https://developers.openai.com/api/docs/guides/agents/sdk>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

16. Tool Design and Agent-Computer Interfaces

Good tools reduce reasoning burden

Why this exists

Agent reliability often depends more on tool interface quality than on adding more model intelligence. Ambiguous names, huge schemas and noisy outputs create avoidable reasoning failures.

Agent-computer interface



Figure: Agent-computer interface

Three concepts to retain

Narrow responsibility: Each tool should do one coherent thing.

Typed contract: Parameters, outputs and errors should be machine-checkable.

Observability: Every tool call needs identity, timing, result and side-effect status.

Tool Design and Agent-Computer Interfaces - Mechanics

How it works

1. Name tools by intent.
2. Use constrained enums and required fields.
3. Return concise machine-readable results.
4. Classify transient vs permanent errors.
5. Expose idempotency/operation IDs for side effects.

Worked example

“manage_user” with 25 optional fields is harder than read_user, update_user_profile and disable_user. Narrow tools reduce accidental side effects and make policy enforcement simpler.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Tool Design and Agent-Computer Interfaces - Failure Modes

Reason through these failures

- **Tools that return entire database tables.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Parameters described only in prose.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Retrying non-idempotent actions without operation IDs.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Giving the agent tools it never needs.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in tool design and agent-computer interfaces is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Tool Design and Agent-Computer Interfaces - Build and Recall

Lab

1. Take one broad internal API and split it into three narrow tools.
2. Add JSON schema and error codes.
3. Write 20 tool-selection evals.
4. Measure wrong-tool and malformed-argument rates.

Recall without notes

- Explain why tool design and agent-computer interfaces exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - Writing effective tools for agents - <https://www.anthropic.com/engineering/writing-tools-for-agents>

Anthropic - Building effective agents - <https://www.anthropic.com/engineering/building-effective-agents>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

17. ReAct, Planning and Decomposition

Reason enough to choose actions, not to create ceremony

Why this exists

Planning is valuable when tasks have dependencies, uncertainty or parallel branches. It becomes overhead when the task is short and deterministic.

Plan - act - observe - replan



Figure: Plan - act - observe - replan

Three concepts to retain

ReAct loop: Interleave reasoning, action and observation.

Plan: A tentative decomposition, not an immutable script.

Replan: Update the plan when observations invalidate assumptions.

ReAct, Planning and Decomposition - Mechanics

How it works

1. Identify the goal and constraints.
2. Decompose only where useful.
3. Execute one or more steps.
4. Compare observations with expected progress.
5. Replan or stop.

Worked example

A research task can split into independent source discovery branches, then merge findings. A two-field data extraction task should not spend a model call generating a plan first.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

ReAct, Planning and Decomposition - Failure Modes

Reason through these failures

- **Planning every trivial request.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Never revising a plan after evidence changes.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Generating dozens of vague subtasks.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Parallelizing tasks that mutate the same state.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in react, planning and decomposition is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

ReAct, Planning and Decomposition - Build and Recall

Lab

1. Implement a planner that outputs at most five typed steps.
2. Allow two independent steps to run in parallel.
3. Inject a failed tool result and require replan.
4. Compare latency against a direct agent loop.

Recall without notes

- Explain why react, planning and decomposition exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

ReAct paper - <https://arxiv.org/abs/2210.03629>

Anthropic - Building effective agents - <https://www.anthropic.com/engineering/building-effective-agents>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

18. Context Engineering

Give the model the right information, not all information

Why this exists

Context is a finite working set. Loading every tool schema, old message and raw API result can increase cost while making important evidence harder to use.

Context as a budgeted working set

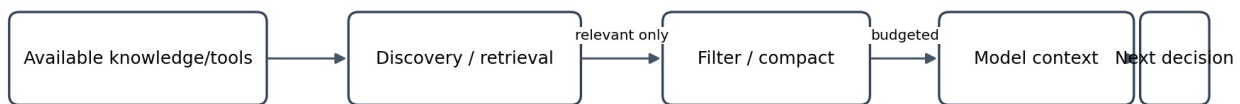


Figure: Context as a budgeted working set

Three concepts to retain

Working set: Only information needed for the next decisions.

Progressive disclosure: Retrieve tools/data when they become relevant.

Compaction: Reduce old trajectory while preserving machine-relevant state.

Context Engineering - Mechanics

How it works

1. Budget context by category.
2. Retrieve rather than preload large corpora/tool catalogs.
3. Filter large tool results outside the model.
4. Compact stale trajectory.
5. Reserve headroom for unexpected work.

Worked example

A connector exposes 400 tools at 250 tokens each: 100k tokens of schema before the task begins. Dynamic discovery that surfaces 10 tools reduces this to roughly 2.5k tokens.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Context Engineering - Failure Modes

Reason through these failures

- **Treating context window as free storage.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Summaries that lose IDs/state needed to continue.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Returning raw 50k-row API results to the model.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No reserve for tool responses or final synthesis.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in context engineering is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Context Engineering - Build and Recall

Lab

1. Measure token usage of an agent with all tools loaded.
2. Add tool discovery.
3. Filter one large API result in code.
4. Compare success, cost and latency.

Recall without notes

- Explain why context engineering exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - Effective context engineering for AI agents -

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Anthropic - Code execution with MCP - <https://www.anthropic.com/engineering/code-execution-with-mcp>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

19. Memory, State and Artifacts

Do not use chat history as your database

Why this exists

Long-lived AI systems need explicit separation between transient context, durable execution state, reusable memory and authoritative artifacts.

Separate context, state, memory and artifacts

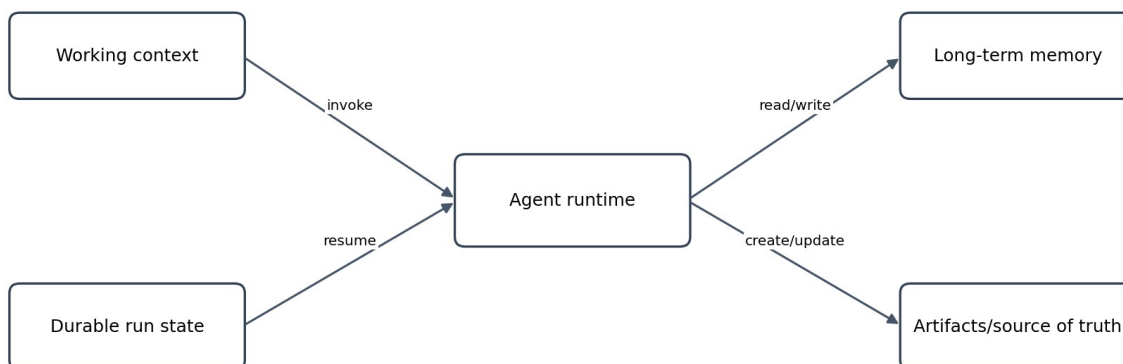


Figure: Separate context, state, memory and artifacts

Three concepts to retain

Context: What the current model invocation sees.

Execution state: Objective, completed steps, IDs and statuses required to resume.

Memory: Reusable facts or preferences across future runs.

Artifact: A durable output such as code, report or structured record.

Memory, State and Artifacts - Mechanics

How it works

1. Persist state outside the context window.
2. Store only memories that improve future decisions.
3. Keep provenance and timestamps.
4. Treat generated artifacts as versioned data.
5. Rehydrate the smallest necessary context when resuming.

Worked example

A coding agent should persist git commit, failing tests and feature status as durable state. A prose summary saying “worked on auth” is not enough to resume safely.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Memory, State and Artifacts - Failure Modes

Reason through these failures

- **Appending every conversation forever.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Confusing summary with authoritative state.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Storing stale preferences with no version/provenance.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Allowing memory to override current source-of-truth data.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in memory, state and artifacts is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Memory, State and Artifacts - Build and Recall

Lab

1. Define RunState as JSON.
2. Persist objective, step statuses, artifacts and last checkpoint.
3. Kill the process mid-run.
4. Restart and resume without replaying the entire transcript.

Recall without notes

- Explain why memory, state and artifacts exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - Effective harnesses for long-running agents -

<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

OpenAI Agents SDK guide - <https://developers.openai.com/api/docs/guides/agents/sdk>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

20. Multi-Agent Systems

Parallelism and specialization without coordination chaos

Why this exists

Multiple agents can expand effective context and parallelize independent work, but they also introduce duplicated effort, communication cost and shared-state hazards.

Orchestrated multi-agent architecture

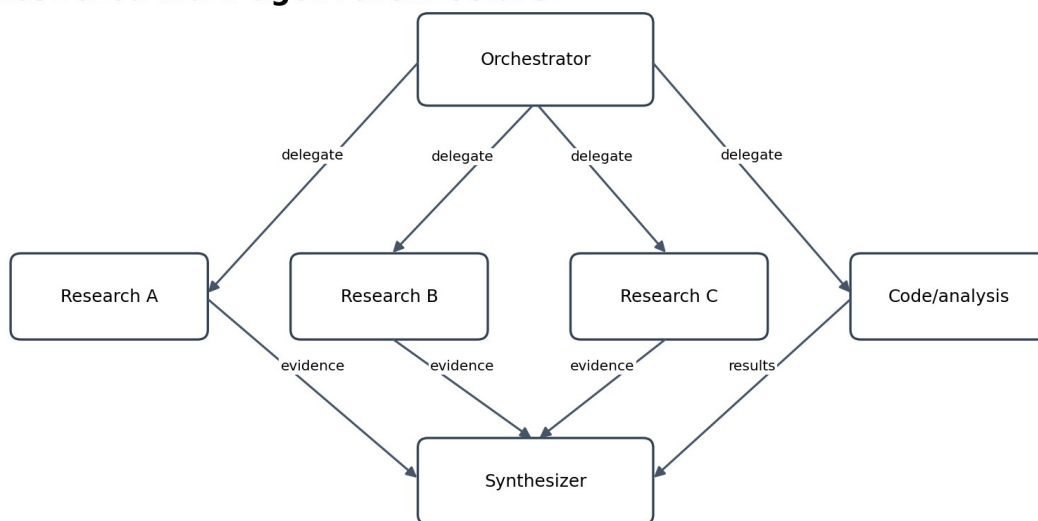


Figure: Orchestrated multi-agent architecture

Three concepts to retain

Orchestrator-worker: One controller delegates bounded tasks.

Parallel specialists: Independent agents explore separate sources or hypotheses.

Shared-state discipline: Avoid uncoordinated concurrent mutation.

Multi-Agent Systems - Mechanics

How it works

1. Use one agent until limits are demonstrated.
2. Partition work by independent context or skill.
3. Give workers narrow outputs.
4. Aggregate results through a controlled synthesizer.
5. Evaluate contribution, not number of agents.

Worked example

A deep-research system can ask separate workers to investigate market, technical and regulatory evidence, then synthesize. Five agents discussing the same paragraph often add cost without new information.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Multi-Agent Systems - Failure Modes

Reason through these failures

- **Agents recursively spawning agents.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **All workers retrieving the same sources.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Concurrent edits to one file with no ownership.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No attribution of which worker improved outcome.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in multi-agent systems is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Multi-Agent Systems - Build and Recall

Lab

1. Run one research task with 1, 3 and 5 workers.
2. Assign non-overlapping source scopes.
3. Measure coverage, duplication, cost and wall time.
4. Add a synthesizer with citation deduplication.

Recall without notes

- Explain why multi-agent systems exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - How we built our multi-agent research system -

<https://www.anthropic.com/engineering/multi-agent-research-system>

Anthropic - Building a C compiler with a team of parallel Claudes -

<https://www.anthropic.com/engineering/building-c-compiler>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

21. Durable Long-Running Agents

Hours of work require a harness, not one giant prompt

Why this exists

Long tasks outlive context windows, connections and processes. Reliability comes from durable checkpoints, explicit progress and verifiable intermediate artifacts.

Durable agent run

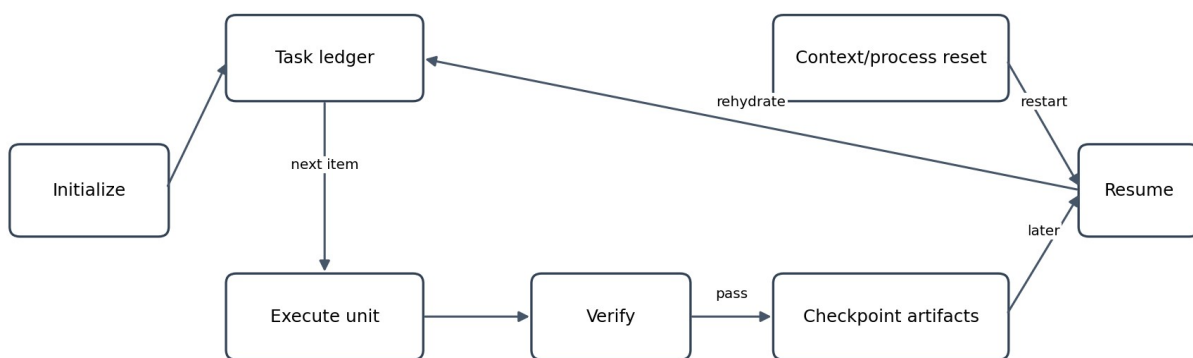


Figure: Durable agent run

Three concepts to retain

Checkpoint: A recoverable boundary with persisted state.

Progress ledger: Structured record of completed and remaining work.

Session handoff: Each new context can reconstruct current state from artifacts.

Durable Long-Running Agents - Mechanics

How it works

1. Initialize workspace and task ledger.
2. Choose a bounded unit of work.
3. Execute and verify it.
4. Persist artifact + status + commit/checkpoint.
5. Resume from persisted truth after context reset or crash.

Worked example

Anthropic describes long-running coding harnesses that use feature lists, progress notes, git history and self-verification so new sessions can continue incrementally rather than attempting a one-shot build.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Durable Long-Running Agents - Failure Modes

Reason through these failures

- **Declaring the whole project done prematurely.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Context compaction without explicit progress state.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No baseline test when a new session starts.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Marking tasks complete before verification.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in durable long-running agents is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Durable Long-Running Agents - Build and Recall

Lab

1. Create a 20-feature JSON ledger.
2. Make an agent select one unfinished feature per run.
3. Require tests before status=done.
4. Terminate the process and verify a new process can resume.

Recall without notes

- Explain why durable long-running agents exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - Effective harnesses for long-running agents -

<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

PART IV - EVALS AND OBSERVABILITY

Build the mental model, then test the failure modes.

22. Evaluation Datasets and Failure Taxonomy

Turn “seems good” into measurable capability

Why this exists

A useful eval suite reflects the tasks and failure modes your application actually encounters. Random prompts and generic benchmarks rarely protect a product from regressions.

Eval dataset flywheel



Figure: Eval dataset flywheel

Three concepts to retain

Core tasks: Representative high-frequency workflows.

Regression cases: Real historical failures that should never reappear.

Slices: Meaningful subgroups that expose hidden weaknesses.

Evaluation Datasets and Failure Taxonomy - Mechanics

How it works

1. Start with manual error analysis.
2. Name failure categories.
3. Build small high-value task sets.
4. Define graders and success thresholds.
5. Add production failures back into the suite.

Worked example

Overall task success can rise from 91% to 94% while document-upload questions fall from 95% to 78%. Slice-level reporting prevents the aggregate from hiding a critical regression.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Evaluation Datasets and Failure Taxonomy - Failure Modes

Reason through these failures

- **Writing a giant rubric before inspecting errors.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using one average score.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Allowing test set leakage into prompt tuning.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Never refreshing evals after product behavior changes.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in evaluation datasets and failure taxonomy is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Evaluation Datasets and Failure Taxonomy - Build and Recall

Lab

1. Review 100 traces manually.
2. Create a failure taxonomy.
3. Build core.jsonl, regressions.jsonl and edge_cases.jsonl.
4. Run current vs candidate and produce slice deltas.

Recall without notes

- Explain why evaluation datasets and failure taxonomy exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Hamel Husain - Putting Evals Into Production -

<https://hamel.dev/notes/llm/ai-product-engineering/evals-production.html>

Anthropic - Demystifying evals for AI agents - <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

23. LLM-as-Judge Calibration

Evaluate the evaluator

Why this exists

LLM judges make evaluation scalable, but they can be biased, inconsistent or systematically blind to the very failures you care about.

Judge calibration

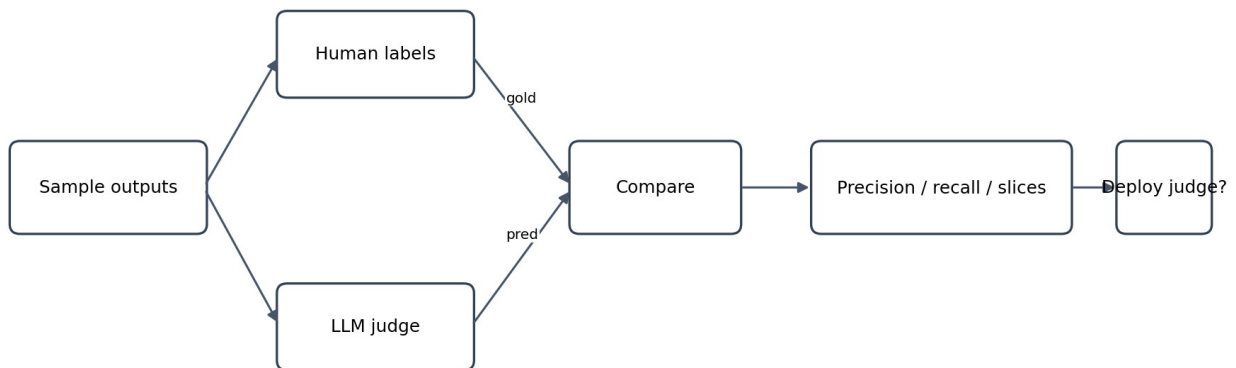


Figure: Judge calibration

Three concepts to retain

Human gold labels: Reference judgments from domain experts.

Precision/recall: Different judge errors have different product consequences.

Calibration: Validate the judge on the same distribution it will monitor.

LLM-as-Judge Calibration - Mechanics

How it works

1. Define the rubric with examples.
2. Collect double-labeled human data.
3. Resolve disagreements.
4. Run judge predictions.
5. Measure confusion matrix and slice performance.

Worked example

A hallucination detector with 95% overall accuracy but 61% recall misses nearly four in ten hallucinations. Accuracy alone can hide an unsafe evaluator.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

LLM-as-Judge Calibration - Failure Modes

Reason through these failures

- **Treating LLM judge score as ground truth.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No human agreement study.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using a judge whose rubric differs from product requirements.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Changing judge model without revalidation.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in llm-as-judge calibration is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

LLM-as-Judge Calibration - Build and Recall

Lab

1. Label 100 examples with two humans.
2. Measure human agreement.
3. Run an LLM judge.
4. Compute precision, recall and false-negative rate.

Recall without notes

- Explain why llm-as-judge calibration exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Hamel Husain - Putting Evals Into Production -

<https://hamel.dev/notes/llm/ai-product-engineering/evals-production.html>

Anthropic - Demystifying evals for AI agents - <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

24. Agent Trajectory Evaluation

The final answer can be right for the wrong reasons

Why this exists

Tool-using agents can accidentally succeed after bad actions, excessive retries or unsafe paths. Production evals should inspect outcome and trajectory.

Outcome plus trajectory evaluation

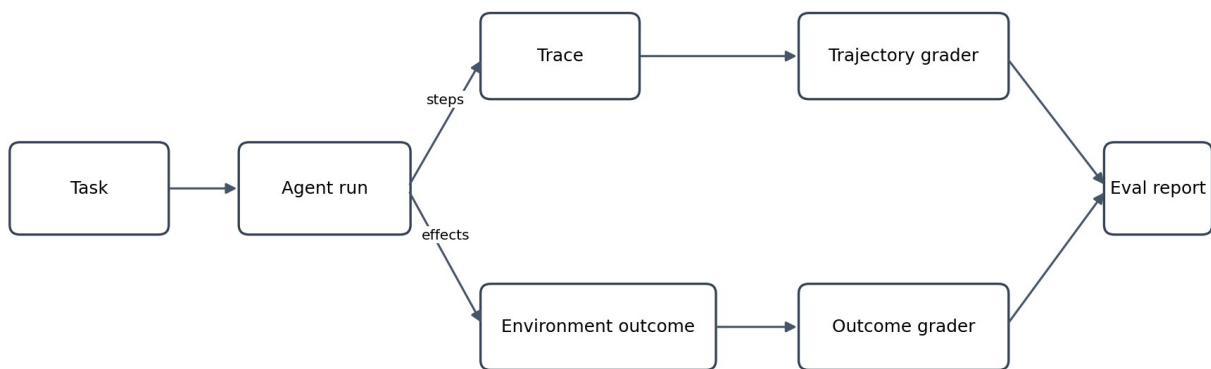


Figure: Outcome plus trajectory evaluation

Three concepts to retain

Transcript: Sequence of model calls, tool calls and observations.

Outcome: Actual end state in the environment.

Trajectory rubric: Rules for acceptable paths: tools, steps, handoffs and side effects.

Agent Trajectory Evaluation - Mechanics

How it works

1. Record the full trace.
2. Grade objective end-state separately.
3. Check tool selection and arguments.
4. Measure steps, retries, cost and unsafe actions.
5. Run multiple trials for stochastic workflows.

Worked example

A booking agent may say “reservation confirmed” even when no database row exists. Outcome grading checks the environment rather than trusting final prose.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Agent Trajectory Evaluation - Failure Modes

Reason through these failures

- **Scoring only final response text.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Ignoring duplicate side effects.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **One trial per task.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Rewarding shorter traces even when verification was skipped.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in agent trajectory evaluation is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Agent Trajectory Evaluation - Build and Recall

Lab

1. Create 20 tool tasks with a mock environment.
2. Record traces.
3. Grade both final state and allowed tool sequence.
4. Run each task three times and report pass@1 and variance.

Recall without notes

- Explain why agent trajectory evaluation exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenAI - Agent evals - <https://developers.openai.com/api/docs/guides/agent-evals>

Anthropic - Demystifying evals for AI agents - <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

25. Observability and Tracing

Reconstruct what one AI run actually did

Why this exists

AI failures span models, retrieval, tools and external systems. Without correlated traces, debugging becomes guesswork across separate logs.

One agent trace

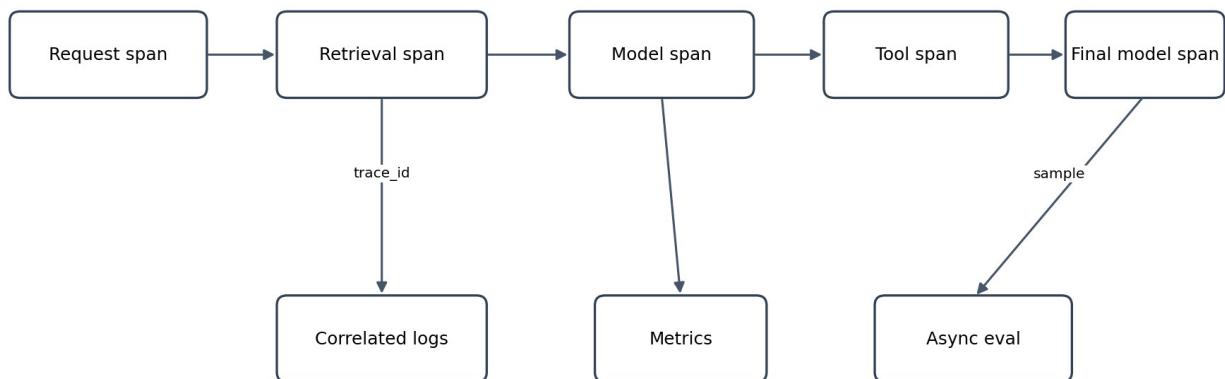


Figure: One agent trace

Three concepts to retain

Trace: End-to-end execution path.

Span: One timed unit of work such as model, tool or retrieval call.

Attributes: Model, tokens, tool name, latency, error class and correlation IDs.

Observability and Tracing - Mechanics

How it works

1. Create a run/trace ID at ingress.
2. Create child spans for model, retrieval and tool operations.
3. Attach structured attributes.
4. Correlate logs and metrics with trace IDs.
5. Sample expensive traces intelligently.

Worked example

A 14-second answer may contain 300 ms retrieval, 10 seconds waiting on one tool, and 3.7 seconds model time. A trace immediately reveals the true bottleneck.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Observability and Tracing - Failure Modes

Reason through these failures

- **Logging prompts without privacy controls.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No correlation between tool logs and agent run.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **High-cardinality labels in metrics.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Sampling away all rare failures.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in observability and tracing is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Observability and Tracing - Build and Recall

Lab

1. Instrument a small RAG agent with OpenTelemetry.
2. Create spans for retrieval, model and tool.
3. Attach token counts and error types.
4. Inspect one slow trace and one failed trace.

Recall without notes

- Explain why observability and tracing exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenTelemetry documentation - <https://opentelemetry.io/docs/>

OpenAI - Tracing - <https://developers.openai.com/api/docs/guides/agents-api/tracing>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

26. Cost Engineering

Measure cost per successful task

Why this exists

Per-token pricing is only one layer. Agent cost depends on the number of calls, tool loops, retrieved context, retries, model routing and success rate.

Where AI cost accumulates

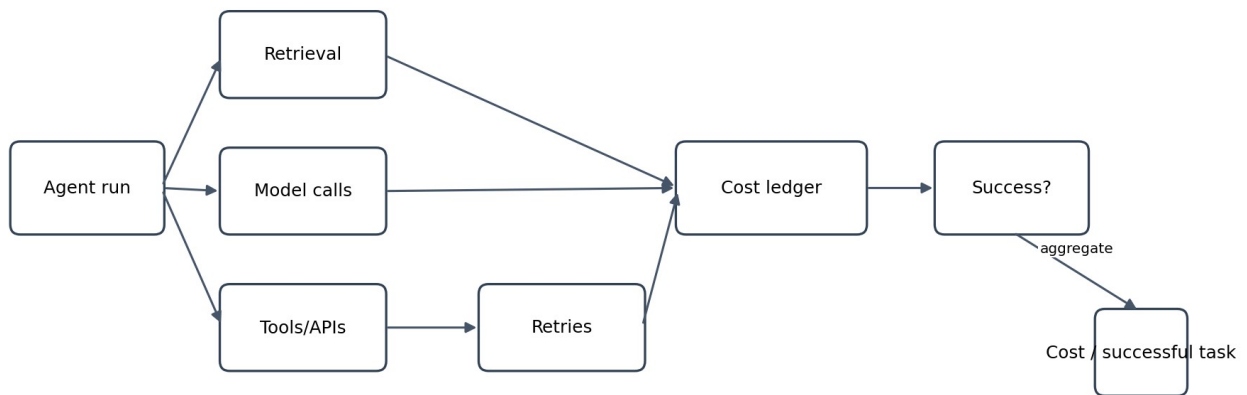


Figure: Where AI cost accumulates

Three concepts to retain

Cost/run: All inference and tool costs incurred by one run.

Cost/success: Total spend divided by successfully completed tasks.

Marginal value: A component should justify the extra cost it adds.

Cost Engineering - Mechanics

How it works

1. Record tokens and prices per model call.
2. Track tool and infrastructure cost when material.
3. Attribute retries to the originating run.
4. Compute cost by task slice.
5. Use ablations to test whether expensive components improve outcomes.

Worked example

System A costs \$0.04/run at 70% success, so cost/success is about \$0.057. System B costs \$0.06/run at 95% success, about \$0.063/success. B costs more but may deliver much higher product value.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Cost Engineering - Failure Modes

Reason through these failures

- **Optimizing average token cost while retries explode.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Ignoring failed-run spend.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Using premium models for deterministic steps.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No per-user/per-task budgets.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in cost engineering is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Cost Engineering - Build and Recall

Lab

1. Instrument a 100-run benchmark.
2. Calculate cost/run and cost/success.
3. Remove one optional verifier and rerun.
4. Build a Pareto table for quality, latency and cost.

Recall without notes

- Explain why cost engineering exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Chip Huyen - Building LLM applications for production - <https://huyenchip.com/2023/04/11/llm-engineering.html>

OpenAI pricing - <https://openai.com/api/pricing/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

PART V - PRODUCTION AGENT SYSTEMS

Build the mental model, then test the failure modes.

27. Latency Engineering

Optimize the critical path, not random components

Why this exists

User-perceived latency is the sum and maximum of dependent stages. Parallelism, output length, model choice and tool behavior often matter more than micro-optimizing one API call.

Latency critical path

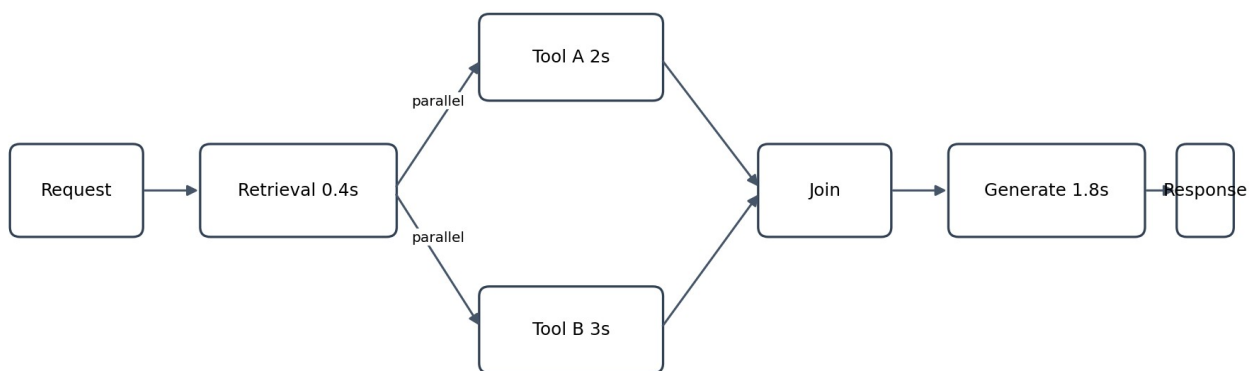


Figure: Latency critical path

Three concepts to retain

TTFT: Time until first generated token.

Critical path: The dependent sequence that determines wall-clock latency.

Parallel branch: Independent work can run concurrently, reducing total time to the maximum branch duration.

Latency Engineering - Mechanics

How it works

1. Build a latency waterfall from traces.
2. Separate queue, retrieval, prefill, decode and tools.
3. Parallelize independent branches.
4. Generate fewer tokens.
5. Use deadlines to stop optional work.

Worked example

If retrieval=0.4s, tool A=2s and tool B=3s are independent, sequential execution costs 5.4s before generation. Parallel tools reduce that portion to roughly 3.4s.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Latency Engineering - Failure Modes

Reason through these failures

- **Optimizing median while p99 is dominated by one tool.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Parallelizing dependent operations.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Long explanations when a structured short answer works.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Retries that ignore remaining deadline.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in latency engineering is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Latency Engineering - Build and Recall

Lab

1. Trace 50 requests.
2. Build p50/p95 waterfall by span.
3. Parallelize two independent calls.
4. Compare end-to-end p95 before/after.

Recall without notes

- Explain why latency engineering exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenAI - Latency optimization - <https://developers.openai.com/api/docs/guides/latency-optimization>

Chip Huyen - Building LLM applications for production - <https://huyenchip.com/2023/04/11/llm-engineering.html>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

28. SLOs and Resource Budgets for AI

Define healthy enough before incidents define it for you

Why this exists

Production AI needs explicit limits on latency, cost, steps and acceptable failure. Otherwise a run can remain technically alive while consuming unbounded resources or missing user expectations.

Budgeted agent run

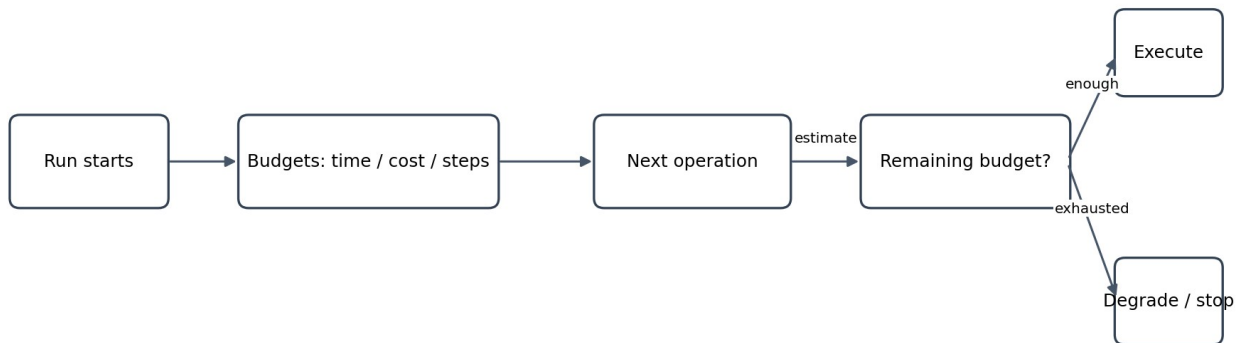


Figure: Budgeted agent run

Three concepts to retain

SLI: A measurable user-facing success signal.

SLO: Target for that indicator over a window.

Run budget: Hard limits on time, cost, tokens, steps and tool calls.

SLOs and Resource Budgets for AI - Mechanics

How it works

1. Define user journeys.
2. Choose good-event criteria.
3. Set SLOs based on product need.
4. Propagate request deadlines.
5. Terminate or degrade runs when budgets are exhausted.

Worked example

If “research answer completed within 20s” is the SLO, giving every component a 20s timeout violates the end-to-end objective. Each operation must consume from one remaining deadline.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
if now() >= run.deadline:
    return degrade_or_stop(run)
if run.cost + next_cost > run.max_cost:
    return cheaper_path(run)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

SLOs and Resource Budgets for AI - Failure Modes

Reason through these failures

- **CPU utilization as the only SLI.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No distinction between interactive and background jobs.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Model retry loop after deadline is already impossible.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **100% reliability targets with no error budget.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in slo's and resource budgets for ai is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

SLOs and Resource Budgets for AI - Build and Recall

Lab

1. Define one availability and one latency SLI.
2. Add `deadline_remaining` to `RunState`.
3. Create `MAX_COST` and `MAX_STEPS`.
4. Inject a slow dependency and verify graceful termination.

Recall without notes

- Explain why slo and resource budgets for ai exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Google SRE Workbook - Alerting on SLOs - <https://sre.google/workbook/alerting-on-slos/>

OpenAI - Latency optimization - <https://developers.openai.com/api/docs/guides/latency-optimization>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

29. Retries, Fallbacks and Graceful Degradation

Useful partial service beats uncontrolled collapse

Why this exists

Provider errors and slow tools are normal. Reliability comes from classifying failures, bounding retries and intentionally reducing optional work under pressure.

Graceful degradation ladder

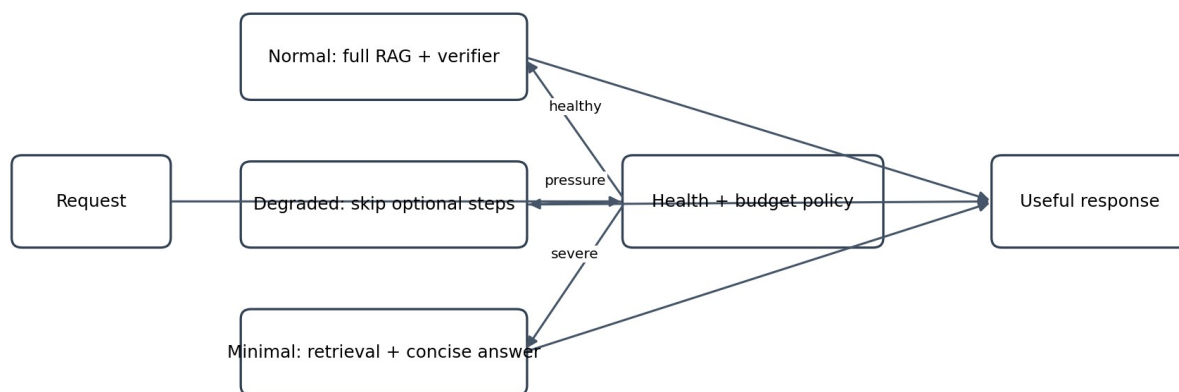


Figure: Graceful degradation ladder

Three concepts to retain

Transient error: May succeed later: 429, 503, network reset.

Permanent error: Retrying unchanged request is pointless: invalid schema, forbidden action.

Degradation ladder: Predefined lower-cost/lower-feature execution modes.

Retries, Fallbacks and Graceful Degradation - Mechanics

How it works

1. Classify error.
2. Honor Retry-After when provided.
3. Use exponential backoff with jitter.
4. Respect end-to-end deadline and retry budget.
5. Fall back only to paths that pass the same eval contract.

Worked example

If a reranker slows from 300 ms to 4 s, a 12 s interactive SLO may be better served by skipping reranking than by retrying it twice and timing out the entire answer.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
for attempt in range(MAX_RETRIES):
    try: return call()
    except TransientError:
        sleep(backoff(attempt) + jitter())
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Retries, Fallbacks and Graceful Degradation - Failure Modes

Reason through these failures

- **Retry every failure.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Immediate retry storms.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Fallback to a model with incompatible tool behavior.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No signal showing users are receiving degraded service.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in retries, fallbacks and graceful degradation is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Retries, Fallbacks and Graceful Degradation - Build and Recall

Lab

1. Inject 429, 503 and invalid-schema errors.
2. Implement bounded backoff.
3. Add NORMAL, DEGRADED and MINIMAL execution tiers.
4. Measure useful-response rate and p95.

Recall without notes

- Explain why retries, fallbacks and graceful degradation exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenAI - Latency optimization - <https://developers.openai.com/api/docs/guides/latency-optimization>

AWS Builders Library - Timeouts, retries and backoff with jitter - <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

30. Canary Releases and AI Regression Gates

Limit how many users discover your mistake

Why this exists

Offline evals cannot predict every production behavior. Safe releases combine regression suites with progressive exposure and rollback criteria.

AI canary release

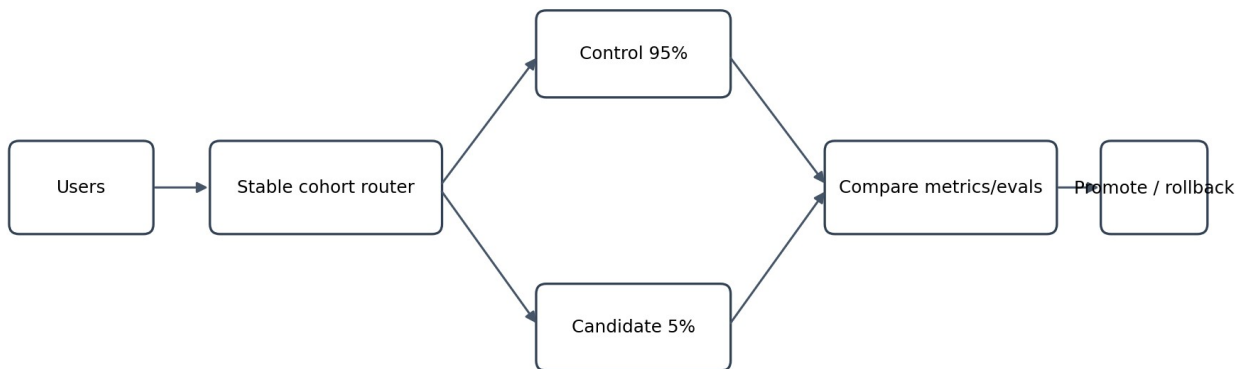


Figure: AI canary release

Three concepts to retain

Control: Known-good prompt/model/policy.

Candidate: Changed system under evaluation.

Canary: Small stable cohort receiving the candidate.

Release gate: Predefined quality, cost and latency thresholds.

Canary Releases and AI Regression Gates - Mechanics

How it works

1. Run candidate against offline core/regression suites.
2. Shadow or dark-launch where possible.
3. Expose a small deterministic cohort.
4. Compare technical and product metrics.
5. Promote or rollback based on predefined rules.

Worked example

A model upgrade raises overall task success 3 points but cuts uploaded-document accuracy 15 points. A slice-level regression gate should block it even though the average improved.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Canary Releases and AI Regression Gates - Failure Modes

Reason through these failures

- **Full rollout after one benchmark.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Random per-request cohort causing inconsistent sessions.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No state compatibility for rollback.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Changing thresholds after seeing candidate results.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in canary releases and ai regression gates is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Canary Releases and AI Regression Gates - Build and Recall

Lab

1. Create current and candidate prompts.
2. Run offline evals.
3. Route 5% of synthetic users to candidate using `hash(user_id)`.
4. Auto-disable candidate when failure delta crosses threshold.

Recall without notes

- Explain why canary releases and ai regression gates exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Hamel Husain - Putting Evals Into Production -

<https://hamel.dev/notes/llm/ai-product-engineering/evals-production.html>

Martin Fowler - Feature Toggles - <https://martinfowler.com/articles/feature-toggles.html>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

31. Security, Sandboxing and Agent Containment

Reduce blast radius with deterministic boundaries

Why this exists

A capable agent will eventually make a wrong decision or encounter malicious content. Security cannot rely only on the model refusing dangerous actions.

Agent containment boundary

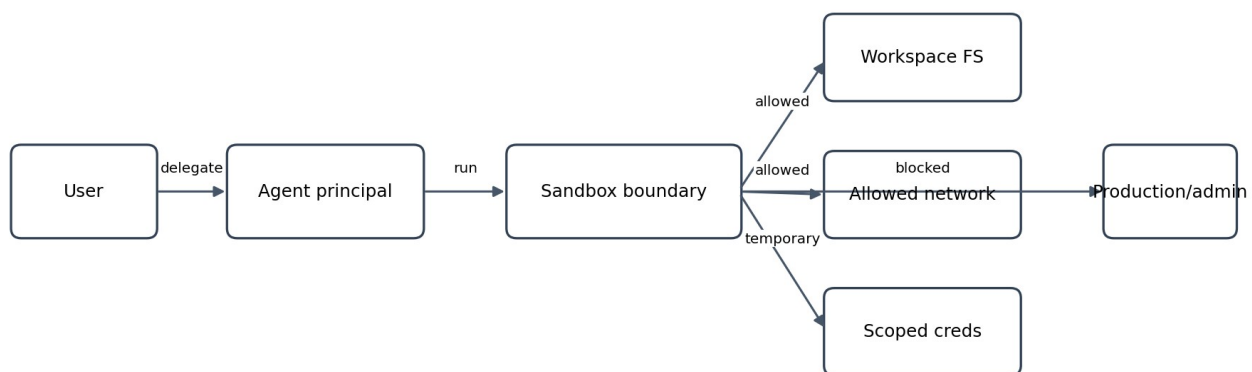


Figure: Agent containment boundary

Three concepts to retain

Least privilege: Agent permissions should be a strict subset of user/admin authority.

Sandbox: Filesystem/process/network boundaries restrict reachable capabilities.

Scoped credential: Short-lived capability tied to one task or session.

Prompt injection surface: Retrieved content can contain adversarial instructions.

Security, Sandboxing and Agent Containment - Mechanics

How it works

1. Create an agent principal with narrow permissions.
2. Deny network by default.
3. Expose only required workspace paths.
4. Keep master credentials outside the sandbox.
5. Gate irreversible actions with policy/human approval.

Worked example

A coding agent can read/write one repository and create a branch while being unable to access ~/.ssh, production credentials or arbitrary outbound hosts. This allows more autonomy inside a smaller blast radius.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
policy = CapabilityPolicy(files=['/workspace'], network=['docs.api'], deny=['~/.ssh'])
run_agent(policy=policy, scoped_token=token)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Security, Sandboxing and Agent Containment - Failure Modes

Reason through these failures

- **Agent inherits developer admin token.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Domain allowlist treated as safe even when endpoint can exfiltrate data.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Permission prompts on every harmless action causing approval fatigue.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Trusting retrieved README text as instructions.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in security, sandboxing and agent containment is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Security, Sandboxing and Agent Containment - Build and Recall

Lab

1. Write a capability manifest.
2. Run agent in a container with mounted project directory only.
3. Block outbound network except one mock API.
4. Test malicious file instructions and unexpected file reads.

Recall without notes

- Explain why security, sandboxing and agent containment exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Anthropic - How we contain Claude across products - <https://www.anthropic.com/engineering/how-we-contain-claude>

OWASP - LLM Prompt Injection - <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

32. Human Approval and Risk-Tiered Autonomy

Escalate by risk and irreversibility, not by fear

Why this exists

Human-in-the-loop is valuable for consequential actions, but asking for approval on everything creates fatigue and destroys agent utility.

Risk-tiered tool execution

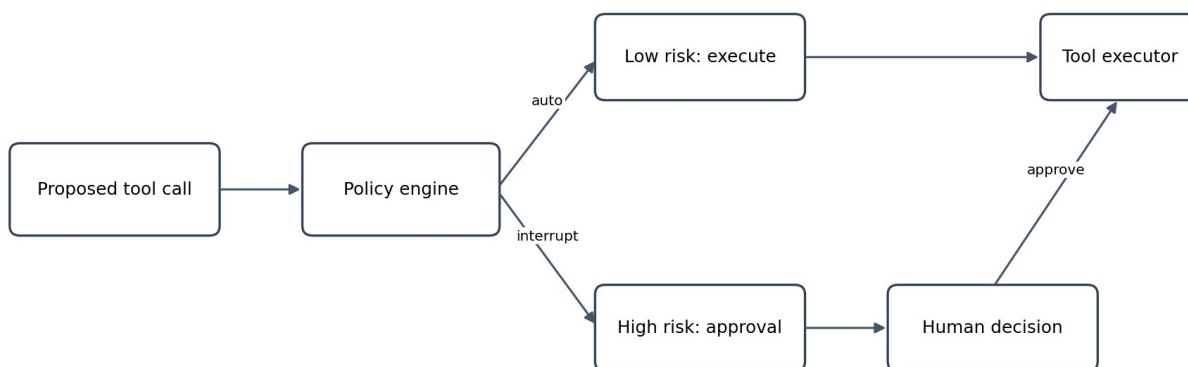


Figure: Risk-tiered tool execution

Three concepts to retain

Risk tier: Potential harm if action is wrong.

Reversibility: How cheaply the action can be undone.

Approval interrupt: Persist state, ask human, then resume the same run.

Human Approval and Risk-Tiered Autonomy - Mechanics

How it works

1. Classify tool actions.
2. Auto-run low-risk reversible operations.
3. Require approval for high-risk/irreversible actions.
4. Persist a resumable approval request.
5. Record approver, decision and exact proposed action.

Worked example

Reading logs is low risk; drafting an email is moderate; sending it externally may require approval; deleting production data or transferring funds requires strong authorization.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Human Approval and Risk-Tiered Autonomy - Failure Modes

Reason through these failures

- **Approval dialog hides what will actually happen.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Agent changes parameters after approval.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Approval state stored only in the live context.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Users click approve repeatedly because prompts are too frequent.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in human approval and risk-tiered autonomy is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Human Approval and Risk-Tiered Autonomy - Build and Recall

Lab

1. Add risk metadata to each tool.
2. Implement policy outside the model.
3. Pause high-risk calls.
4. Resume only after signed approval for the exact arguments.

Recall without notes

- Explain why human approval and risk-tiered autonomy exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenAI - Guardrails and approvals - <https://developers.openai.com/api/docs/guides/agents/guardrails-approvals>

Anthropic - How we contain Claude across products - <https://www.anthropic.com/engineering/how-we-contain-claude>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

33. Verification, Provenance and Evidence-Bearing Outputs

Make correctness cheap to inspect

Why this exists

Generation can become cheap while human verification remains expensive. Strong AI products expose evidence, assumptions and editable units so users do not need to redo the work.

Evidence-bearing answer

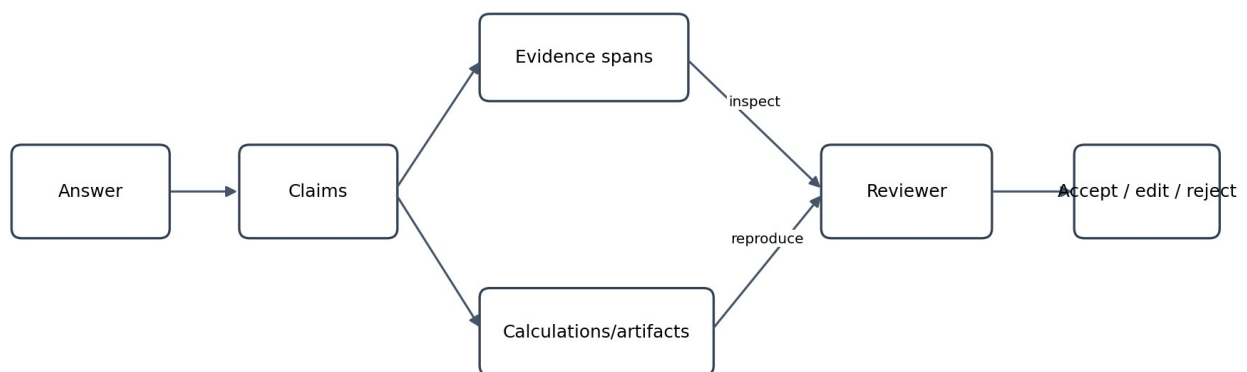


Figure: Evidence-bearing answer

Three concepts to retain

Provenance: Where each important claim came from.

Verification surface: UI/artifact structure that lets users inspect evidence efficiently.

Uncertainty: Explicitly record unresolved or conflicting information.

Verification, Provenance and Evidence-Bearing Outputs - Mechanics

How it works

1. Represent important claims as structured units.
2. Attach source IDs and evidence spans.
3. Expose calculations or deterministic intermediate artifacts.
4. Separate verified facts from inferred synthesis.
5. Design around how domain experts actually check work.

Worked example

A finance agent should not only return “revenue was \$4.2M”; it should link the metric definition, source rows, calculation and unresolved exclusions so a reviewer can establish correctness quickly.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Verification, Provenance and Evidence-Bearing Outputs - Failure Modes

Reason through these failures

- **Citation points to source but not supporting passage.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Showing hidden chain-of-thought instead of useful evidence.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Confidence score with no calibration.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Dumping every intermediate token on the user.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in verification, provenance and evidence-bearing outputs is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Verification, Provenance and Evidence-Bearing Outputs - Build and Recall

Lab

1. Create a claim-evidence JSON output.
2. Have reviewers verify 20 answers with and without evidence links.
3. Measure time-to-verify and reviewer accuracy.
4. Improve the interface based on slow cases.

Recall without notes

- Explain why verification, provenance and evidence-bearing outputs exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Hamel Husain - AI Product Engineering notes - <https://hamel.dev/notes/llm/ai-product-engineering/>

OpenAI - Agent evals - <https://developers.openai.com/api/docs/guides/agent-evals>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

34. Continuous Improvement and Repair Loops

Measured self-improvement, not uncontrolled self-modification

Why this exists

Production agents improve fastest when real traces become labeled data, failures become evals, and proposed changes are benchmarked before deployment.

Closed-loop improvement

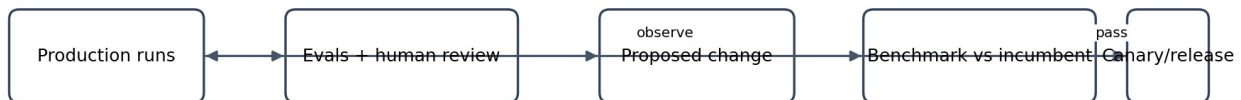


Figure: Closed-loop improvement

Three concepts to retain

Improvement loop: Observe -> evaluate -> propose -> test -> release.

Validator: A reliable external check, ideally deterministic for code/structured tasks.

Incumbent comparison: Candidate must beat the current system under defined metrics.

Continuous Improvement and Repair Loops - Mechanics

How it works

1. Collect representative traces.
2. Score capability-specific failures.
3. Propose prompt/tool/control-flow changes.
4. Run regression and cost tests.
5. Canary the winner and continue monitoring.

Worked example

A coding agent repeatedly fails migrations. Convert those traces into regression tests, improve the tool instructions or validator, and ship only if the new harness improves pass rate without unacceptable cost/latency regressions.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Continuous Improvement and Repair Loops - Failure Modes

Reason through these failures

- **Agent rewrites its own prompt and deploys immediately.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Validator is weaker than generator.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Synthetic improvement does not transfer to production.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No retained incumbent for rollback.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in continuous improvement and repair loops is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Continuous Improvement and Repair Loops - Build and Recall

Lab

1. Create a generate -> test -> repair loop for SQL or code.
2. Bound MAX_REPAIRS=3.
3. Persist each failed attempt.
4. Compare candidate harness with incumbent across regression tasks.

Recall without notes

- Explain why continuous improvement and repair loops exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenAI - Agent evals - <https://developers.openai.com/api/docs/guides/agent-evals>

Anthropic - Demystifying evals for AI agents - <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

35. Prompt Caching and Reuse

Reduce repeated prefix computation and cost

Why this exists

Stable repeated prefixes such as system instructions, tool schemas and large reference context can often be reused by provider-side or application-side caching.

Prompt Caching and Reuse

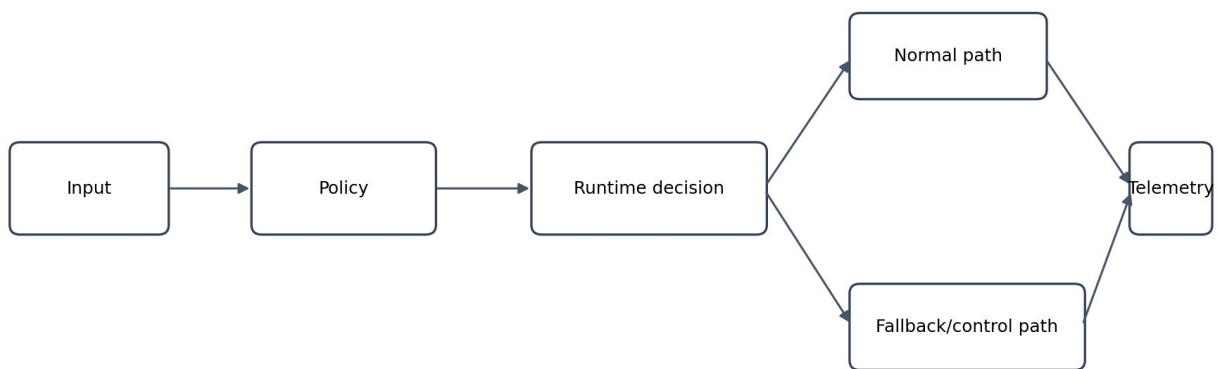


Figure: Prompt Caching and Reuse

Three concepts to retain

Operational contract: Write the expected behavior and failure boundary before implementation.

Explicit state: Make hidden runtime assumptions visible and inspectable.

Tested fallback: A fallback path is production code and must be evaluated.

Prompt Caching and Reuse - Mechanics

How it works

1. Design prompts with stable prefixes, track cache-hit behavior, and avoid assuming every request benefits equally.
2. Measure the relevant metric per user journey.
3. Create a failure-injection test.
4. Record the decision in the run trace.
5. Revisit the policy when models/providers change.

Worked example

Use a small representative workload, deliberately trigger the failure or cache condition, and compare the observed run trace with the intended policy. The purpose is to turn an implicit production assumption into a tested contract.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Prompt Caching and Reuse - Failure Modes

Reason through these failures

- **Policy exists only in a prompt.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No telemetry proving the fallback or cache was used.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **A version change invalidates prior assumptions.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No owner or rollback path.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in prompt caching and reuse is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Prompt Caching and Reuse - Build and Recall

Lab

1. Create one focused experiment for this chapter.
2. Log before/after metrics.
3. Inject one failure case.
4. Write the release criterion.

Recall without notes

- Explain why prompt caching and reuse exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Prompt Caching and Reuse - primary reference - <https://developers.openai.com/api/docs/guides/prompt-caching>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

36. Model and Provider Failure Modes

Design for external AI dependency risk

Why this exists

Hosted model APIs can rate-limit, time out, change behavior or become unavailable. Treat the provider like any critical distributed dependency with isolation and fallback contracts.

Model and Provider Failure Modes

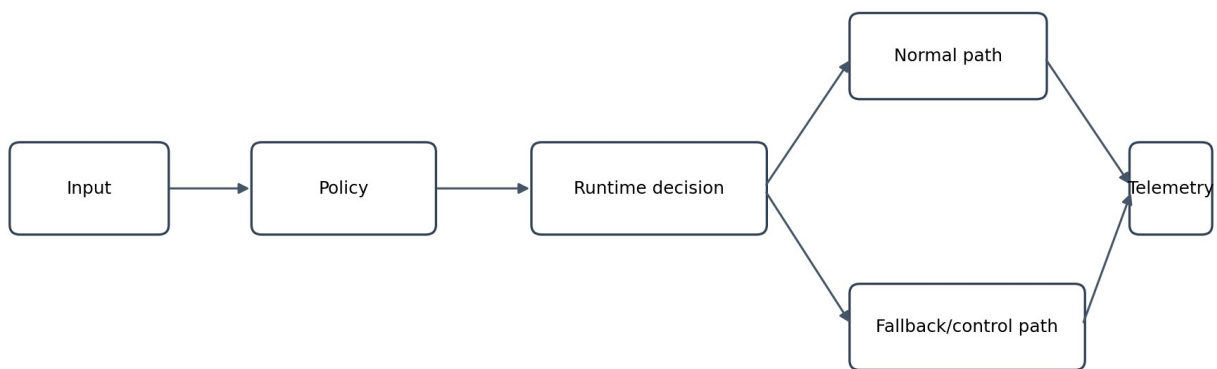


Figure: Model and Provider Failure Modes

Three concepts to retain

Operational contract: Write the expected behavior and failure boundary before implementation.

Explicit state: Make hidden runtime assumptions visible and inspectable.

Tested fallback: A fallback path is production code and must be evaluated.

Model and Provider Failure Modes - Mechanics

How it works

1. Track provider health, classify errors, separate capacity failures from bad requests, and test fallback behavior with the same evals.
2. Measure the relevant metric per user journey.
3. Create a failure-injection test.
4. Record the decision in the run trace.
5. Revisit the policy when models/providers change.

Worked example

Use a small representative workload, deliberately trigger the failure or cache condition, and compare the observed run trace with the intended policy. The purpose is to turn an implicit production assumption into a tested contract.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Model and Provider Failure Modes - Failure Modes

Reason through these failures

- **Policy exists only in a prompt.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No telemetry proving the fallback or cache was used.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **A version change invalidates prior assumptions.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No owner or rollback path.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in model and provider failure modes is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Model and Provider Failure Modes - Build and Recall

Lab

1. Create one focused experiment for this chapter.
2. Log before/after metrics.
3. Inject one failure case.
4. Write the release criterion.

Recall without notes

- Explain why model and provider failure modes exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Model and Provider Failure Modes - primary reference -

<https://developers.openai.com/api/docs/guides/rate-limits>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

37. Prompt Injection and Untrusted Context

Retrieved text is data, not authority

Why this exists

Any external document, webpage or connector result can contain instructions aimed at the model.
Application policy must remain outside untrusted content.

Prompt Injection and Untrusted Context

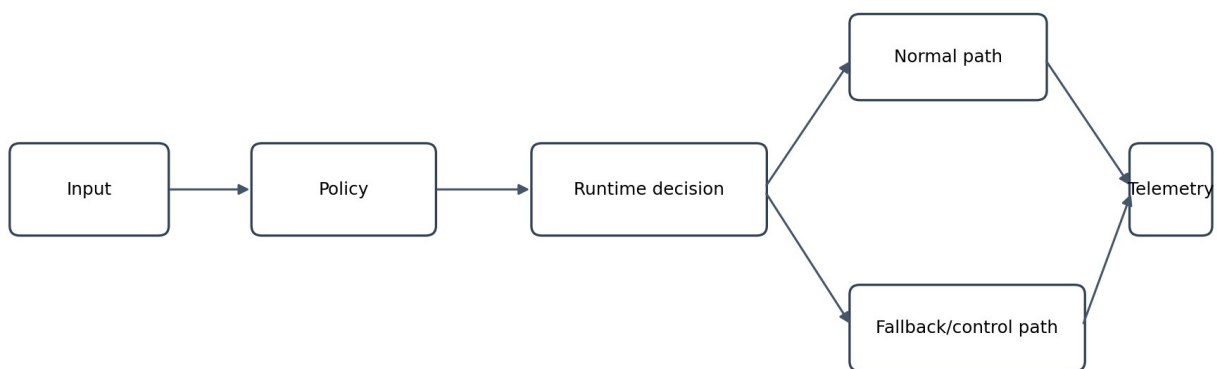


Figure: Prompt Injection and Untrusted Context

Three concepts to retain

Operational contract: Write the expected behavior and failure boundary before implementation.

Explicit state: Make hidden runtime assumptions visible and inspectable.

Tested fallback: A fallback path is production code and must be evaluated.

Prompt Injection and Untrusted Context - Mechanics

How it works

1. Label data/instructions, constrain tools, sanitize where useful, enforce policy outside the model and minimize reachable capability.
2. Measure the relevant metric per user journey.
3. Create a failure-injection test.
4. Record the decision in the run trace.
5. Revisit the policy when models/providers change.

Worked example

Use a small representative workload, deliberately trigger the failure or cache condition, and compare the observed run trace with the intended policy. The purpose is to turn an implicit production assumption into a tested contract.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Prompt Injection and Untrusted Context - Failure Modes

Reason through these failures

- **Policy exists only in a prompt.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No telemetry proving the fallback or cache was used.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **A version change invalidates prior assumptions.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No owner or rollback path.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in prompt injection and untrusted context is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Prompt Injection and Untrusted Context - Build and Recall

Lab

1. Create one focused experiment for this chapter.
2. Log before/after metrics.
3. Inject one failure case.
4. Write the release criterion.

Recall without notes

- Explain why prompt injection and untrusted context exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Prompt Injection and Untrusted Context - primary reference - <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

38. Background Jobs and Resumable Execution

Long work is a state machine

Why this exists

Interactive HTTP connections should not define the lifetime of long-running research or generation jobs.

Background Jobs and Resumable Execution

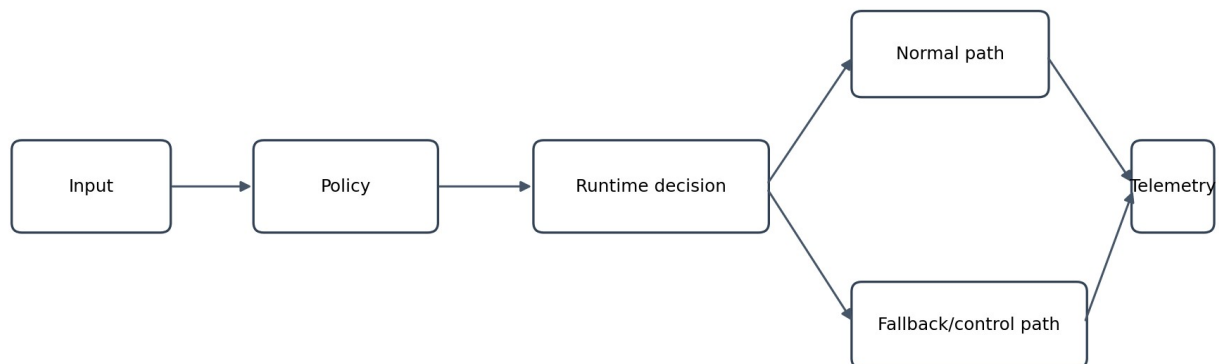


Figure: Background Jobs and Resumable Execution

Three concepts to retain

Operational contract: Write the expected behavior and failure boundary before implementation.

Explicit state: Make hidden runtime assumptions visible and inspectable.

Tested fallback: A fallback path is production code and must be evaluated.

Background Jobs and Resumable Execution - Mechanics

How it works

1. Represent queued/running/waiting/succeeded/failed/cancelled states explicitly and make cancellation/retry idempotent.
2. Measure the relevant metric per user journey.
3. Create a failure-injection test.
4. Record the decision in the run trace.
5. Revisit the policy when models/providers change.

Worked example

Use a small representative workload, deliberately trigger the failure or cache condition, and compare the observed run trace with the intended policy. The purpose is to turn an implicit production assumption into a tested contract.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Background Jobs and Resumable Execution - Failure Modes

Reason through these failures

- **Policy exists only in a prompt.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No telemetry proving the fallback or cache was used.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **A version change invalidates prior assumptions.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No owner or rollback path.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in background jobs and resumable execution is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Background Jobs and Resumable Execution - Build and Recall

Lab

1. Create one focused experiment for this chapter.
2. Log before/after metrics.
3. Inject one failure case.
4. Write the release criterion.

Recall without notes

- Explain why background jobs and resumable execution exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Background Jobs and Resumable Execution - primary reference -

<https://developers.openai.com/api/docs/guides/background>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

39. Production Checklists and Operational Readiness

A launch is a system property

Why this exists

A production checklist catches missing observability, budgets, security controls, rollback and eval coverage before real users expose the gaps.

Production Checklists and Operational Readiness

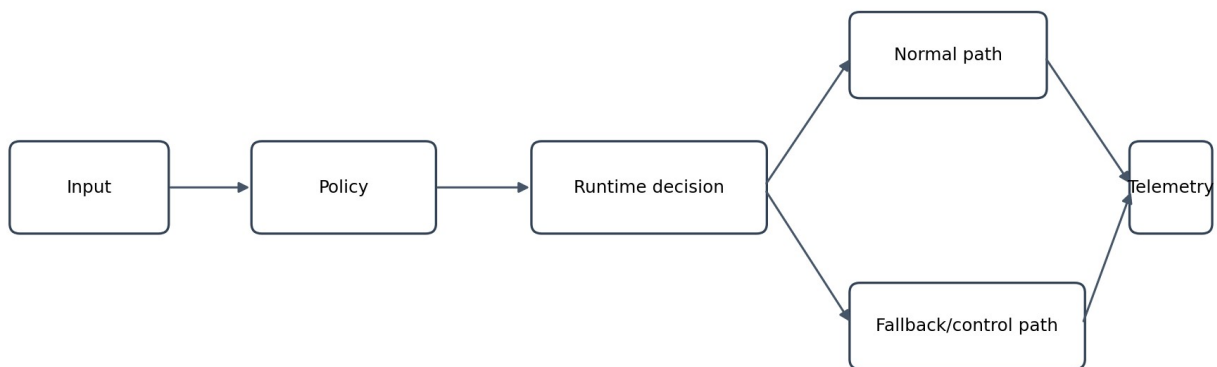


Figure: Production Checklists and Operational Readiness

Three concepts to retain

Operational contract: Write the expected behavior and failure boundary before implementation.

Explicit state: Make hidden runtime assumptions visible and inspectable.

Tested fallback: A fallback path is production code and must be evaluated.

Production Checklists and Operational Readiness - Mechanics

How it works

1. Review quality, latency, cost, safety, state, failure recovery, release policy and ownership before launch.
2. Measure the relevant metric per user journey.
3. Create a failure-injection test.
4. Record the decision in the run trace.
5. Revisit the policy when models/providers change.

Worked example

Use a small representative workload, deliberately trigger the failure or cache condition, and compare the observed run trace with the intended policy. The purpose is to turn an implicit production assumption into a tested contract.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Production Checklists and Operational Readiness - Failure Modes

Reason through these failures

- **Policy exists only in a prompt.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No telemetry proving the fallback or cache was used.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **A version change invalidates prior assumptions.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No owner or rollback path.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in production checklists and operational readiness is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Production Checklists and Operational Readiness - Build and Recall

Lab

1. Create one focused experiment for this chapter.
2. Log before/after metrics.
3. Inject one failure case.
4. Write the release criterion.

Recall without notes

- Explain why production checklists and operational readiness exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

Production Checklists and Operational Readiness - primary reference -

<https://developers.openai.com/api/docs/guides/production-best-practices>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

40. Complete Production Agent Architecture

Putting the pieces together

Why this exists

The final architecture should make model reasoning only one layer inside a controlled runtime that owns identity, state, tools, evidence, budgets, evaluation and operations.

Reference production agent architecture

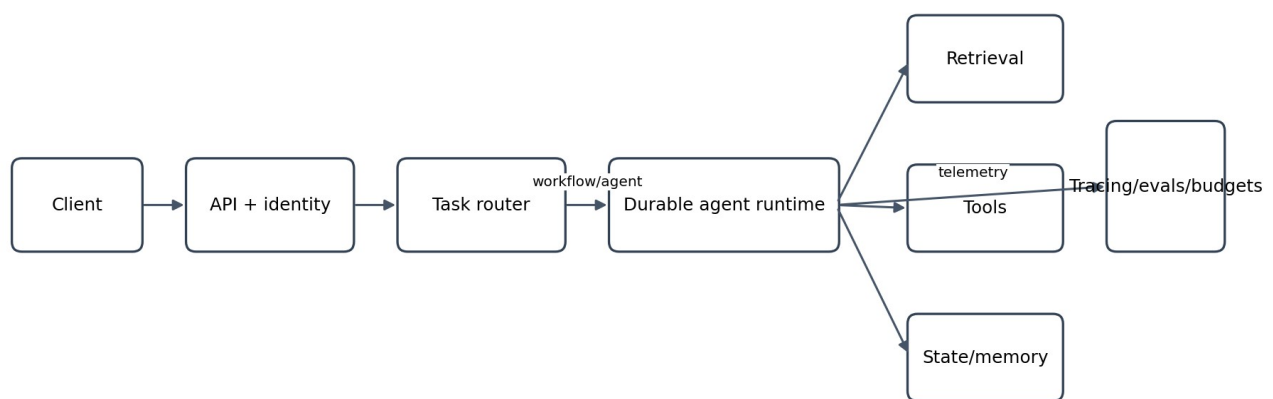


Figure: Reference production agent architecture

Three concepts to retain

Ingress/control: Identity, authorization, routing and policy.

Runtime: State machine, model calls, tools and checkpoints.

Knowledge plane: Retrieval, indexes, documents and permissions.

Operations plane: Tracing, evals, SLOs, release and cost controls.

Complete Production Agent Architecture - Mechanics

How it works

1. Authenticate user and task.
2. Choose workflow/agent path.
3. Create durable RunState and budgets.
4. Retrieve/discover only needed context/tools.
5. Execute with policy, checkpoints and evaluation hooks.
6. Return evidence-bearing result.

Worked example

A research agent can run for 40 minutes while the browser disconnects because the job lives in durable state; it can pause for approval before an external message, resume after a worker crash, and return citations with a trace that supports post-run evaluation.

ENGINEERING LENS: Do not memorize the component name. Identify the state it owns, the information it receives, the decision it makes, and the invariant it must preserve. This makes the design portable across frameworks and vendors.

Implementation sketch

```
result = component(input, policy=policy)
validate(result)
record_trace(input, result, latency_ms, cost)
```

What to measure

- Task success on representative cases.
- p50/p95 latency and the slowest span.
- Cost or resource consumption per successful task.
- Failure rate by explicit error category, not one generic “error”.

Complete Production Agent Architecture - Failure Modes

Reason through these failures

- **One process owns all state.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **No distinction between user permissions and agent permissions.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Retrieval bypasses ACL.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.
- **Observability added only after incidents.** Ask what the user observes, what state may already have changed, and which telemetry would prove the cause.

FAILURE-FIRST QUESTION: If this component becomes slow, unavailable, stale or wrong, does the whole product fail, or can the system detect the condition and degrade safely?

Production design review

- State ownership: identify the source of truth and any cached/derived copies.
- Idempotency: decide whether retrying the operation can duplicate side effects.
- Timeout/deadline: bound waiting and propagate the remaining budget.
- Observability: record enough context to reconstruct one failed run.
- Release: define the regression or canary metric that would block promotion.

Senior-engineer questions

- What assumption in complete production agent architecture is most likely to fail at 10x traffic or 10x task length?
- What would you keep deterministic instead of asking a model to decide?
- What metric could look healthy while the user experience is failing?
- What is the smallest experiment that would falsify your current design?

Complete Production Agent Architecture - Build and Recall

Lab

1. Draw your own architecture before reading the provided diagram.
2. Implement a minimal durable run state.
3. Add one retrieval tool and one side-effecting tool.
4. Add trace, budget, approval and eval hooks.

Recall without notes

- Explain why complete production agent architecture exists in one minute.
- Redraw the chapter architecture from memory.
- Name one correctness failure, one latency failure and one cost failure.
- Explain one design choice you would change for a high-risk enterprise workload.

Primary and implementation sources

OpenAI - Agents SDK - <https://developers.openai.com/api/docs/guides/agents/sdk>

Anthropic - Building effective agents - <https://www.anthropic.com/engineering/building-effective-agents>

OpenTelemetry documentation - <https://opentelemetry.io/docs/>

COMPLETION TEST: You are done with this chapter when you can explain the architecture, predict one failure before it happens, and build the lab without copying the diagram.

PART VI - FIVE PRACTICAL PROJECTS

Learn the architecture by forcing it to fail.

Project 1. Document Intelligence and RAG Engine

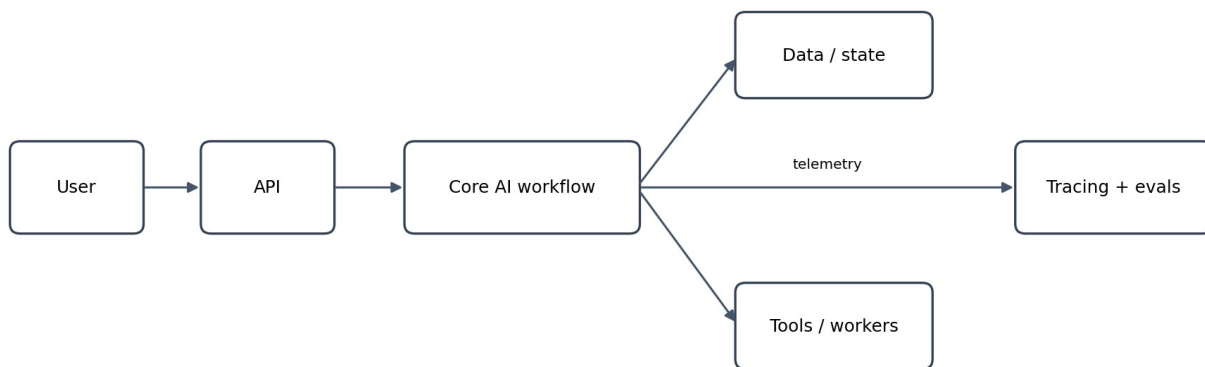
Objective

Build a source-aware technical-document assistant that can answer questions with inspectable evidence.

Suggested stack

Python/FastAPI, PostgreSQL + pgvector, BM25/Elasticsearch or Tantivy, object storage, OpenTelemetry.

Document Intelligence and RAG Engine - reference topology



Document Intelligence and RAG Engine - Milestones

1. Ingest PDFs and HTML with source/page metadata.
2. Implement exact lexical and vector retrieval baselines.
3. Add hybrid retrieval and reranking.
4. Return claim-level citations.
5. Build retrieval and answer eval suites.
6. Add permissions, reindexing and stale-source handling.

RULE: Do not add the next subsystem until you have a measurable baseline for the current one.

Evidence to collect

- Architecture diagram and data-flow diagram.
- Baseline load/eval results.
- A trace from one successful run and one failed run.
- A short decision record for every major technology choice.

Document Intelligence and RAG Engine - Failure Injection

Deliberately create the following conditions and document the observed behavior:

- **OCR/parse failure** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **stale index** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **permission leakage** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **hot tenant** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **missing citation support** - record user-visible effect, telemetry, recovery path and whether the design needs to change.

Minimum reliability bar

- No silent data loss.
- No unbounded retry loop.
- No side effect without an idempotency or approval strategy.
- Every failure is visible in a trace or operational event.

Document Intelligence and RAG Engine - Evaluation

What this project teaches

Embeddings, ANN, chunking, hybrid search, reranking, RAG evaluation, provenance and production ingestion.

Metrics

- Task success / user journey success.
- p50, p95 and timeout rate.
- Cost per successful task.
- Failure distribution by category.
- Human verification time for high-value outputs.

Release exercise

Create a candidate change, run offline regression tests, route 5% of synthetic traffic to it, and define an automatic rollback threshold before you inspect the results.

Document Intelligence and RAG Engine - Architecture Review

Review questions

- Where is the source of truth?
- Which components are derived and can be rebuilt?
- What is the concurrency/capacity bottleneck?
- Which operation is not safe to retry?
- What data must never cross tenant boundaries?
- What is the degraded mode?
- How will you migrate a major storage/model/index change?

Deliverables

- README with architecture and trade-offs.
- One-page runbook.
- Eval/load-test results.
- Failure-injection report.
- Recorded architecture walkthrough.

Document Intelligence and RAG Engine - Final Recall

EXPLAIN IT ALOUD: Describe the request path, state ownership, failure handling, observability and release strategy in under five minutes. If you cannot, simplify or redraw the architecture.

Stretch goals

- Introduce one multi-region or multi-tenant constraint.
- Add a cost ceiling and degradation strategy.
- Simulate a provider or database outage.
- Replace one core dependency and plan a no-downtime migration.

Project 2. Autonomous Coding Agent

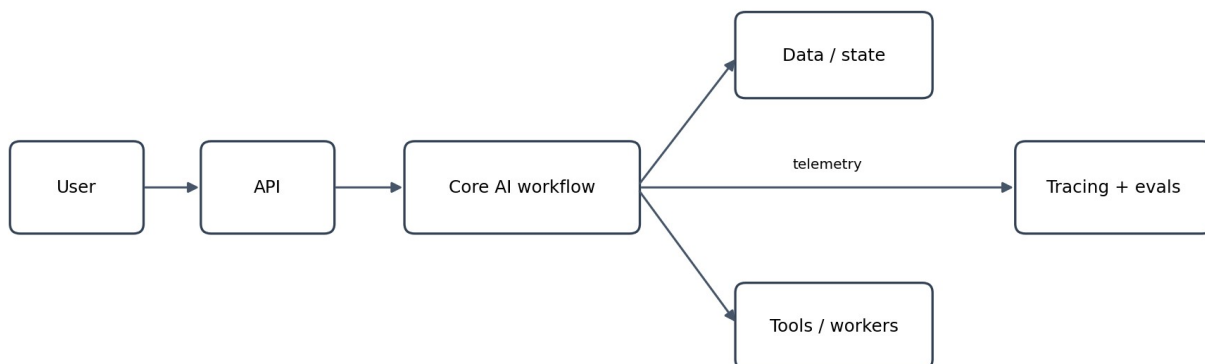
Objective

Build a repository agent that can inspect code, edit files, run tests and make bounded progress across multiple sessions.

Suggested stack

Python/TypeScript agent runtime, git, container sandbox, test runner, SQLite/Postgres run state, OpenTelemetry.

Autonomous Coding Agent - reference topology



Autonomous Coding Agent - Milestones

1. Expose narrow file/search/test tools.
2. Implement task ledger and checkpoints.
3. Require tests before completion.
4. Add repair loop with MAX_REPAIRS.
5. Add sandbox and scoped network.
6. Create regression suite from failed tasks.

RULE: Do not add the next subsystem until you have a measurable baseline for the current one.

Evidence to collect

- Architecture diagram and data-flow diagram.
- Baseline load/eval results.
- A trace from one successful run and one failed run.
- A short decision record for every major technology choice.

Autonomous Coding Agent - Failure Injection

Deliberately create the following conditions and document the observed behavior:

- **premature completion** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **destructive command** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **context loss** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **test flakiness** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **merge conflict** - record user-visible effect, telemetry, recovery path and whether the design needs to change.

Minimum reliability bar

- No silent data loss.
- No unbounded retry loop.
- No side effect without an idempotency or approval strategy.
- Every failure is visible in a trace or operational event.

Autonomous Coding Agent - Evaluation

What this project teaches

Tool design, durable execution, verification, security, repair loops and trajectory evals.

Metrics

- Task success / user journey success.
- p50, p95 and timeout rate.
- Cost per successful task.
- Failure distribution by category.
- Human verification time for high-value outputs.

Release exercise

Create a candidate change, run offline regression tests, route 5% of synthetic traffic to it, and define an automatic rollback threshold before you inspect the results.

Autonomous Coding Agent - Architecture Review

Review questions

- Where is the source of truth?
- Which components are derived and can be rebuilt?
- What is the concurrency/capacity bottleneck?
- Which operation is not safe to retry?
- What data must never cross tenant boundaries?
- What is the degraded mode?
- How will you migrate a major storage/model/index change?

Deliverables

- README with architecture and trade-offs.
- One-page runbook.
- Eval/load-test results.
- Failure-injection report.
- Recorded architecture walkthrough.

Autonomous Coding Agent - Final Recall

EXPLAIN IT ALOUD: Describe the request path, state ownership, failure handling, observability and release strategy in under five minutes. If you cannot, simplify or redraw the architecture.

Stretch goals

- Introduce one multi-region or multi-tenant constraint.
- Add a cost ceiling and degradation strategy.
- Simulate a provider or database outage.
- Replace one core dependency and plan a no-downtime migration.

Project 3. Deep Research Agent

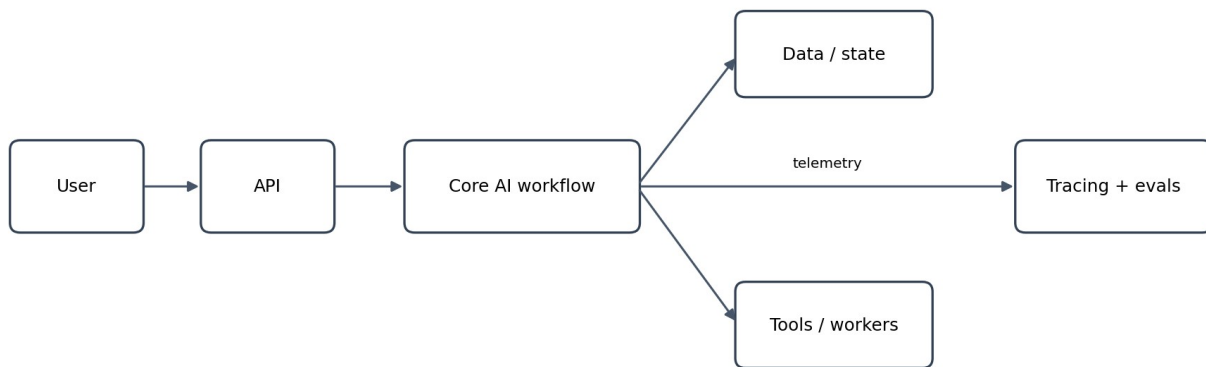
Objective

Build a long-running research system that discovers sources, assigns parallel investigations and produces evidence-bearing synthesis.

Suggested stack

Web/search connectors, queue/job runtime, structured citations, vector store, trace/eval pipeline.

Deep Research Agent - reference topology



Deep Research Agent - Milestones

1. Single-agent research baseline.
2. Source-quality scoring and deduplication.
3. Parallel workers with distinct scopes.
4. Durable job state and resume.
5. Citation/evidence verification.
6. Cost/latency budgets and canary releases.

RULE: Do not add the next subsystem until you have a measurable baseline for the current one.

Evidence to collect

- Architecture diagram and data-flow diagram.
- Baseline load/eval results.
- A trace from one successful run and one failed run.
- A short decision record for every major technology choice.

Deep Research Agent - Failure Injection

Deliberately create the following conditions and document the observed behavior:

- **source duplication** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **citation mismatch** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **runaway search** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **provider timeout** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **worker disagreement** - record user-visible effect, telemetry, recovery path and whether the design needs to change.

Minimum reliability bar

- No silent data loss.
- No unbounded retry loop.
- No side effect without an idempotency or approval strategy.
- Every failure is visible in a trace or operational event.

Deep Research Agent - Evaluation

What this project teaches

Planning, multi-agent orchestration, provenance, long-running state, budgets and evals.

Metrics

- Task success / user journey success.
- p50, p95 and timeout rate.
- Cost per successful task.
- Failure distribution by category.
- Human verification time for high-value outputs.

Release exercise

Create a candidate change, run offline regression tests, route 5% of synthetic traffic to it, and define an automatic rollback threshold before you inspect the results.

Deep Research Agent - Architecture Review

Review questions

- Where is the source of truth?
- Which components are derived and can be rebuilt?
- What is the concurrency/capacity bottleneck?
- Which operation is not safe to retry?
- What data must never cross tenant boundaries?
- What is the degraded mode?
- How will you migrate a major storage/model/index change?

Deliverables

- README with architecture and trade-offs.
- One-page runbook.
- Eval/load-test results.
- Failure-injection report.
- Recorded architecture walkthrough.

Deep Research Agent - Final Recall

EXPLAIN IT ALOUD: Describe the request path, state ownership, failure handling, observability and release strategy in under five minutes. If you cannot, simplify or redraw the architecture.

Stretch goals

- Introduce one multi-region or multi-tenant constraint.
- Add a cost ceiling and degradation strategy.
- Simulate a provider or database outage.
- Replace one core dependency and plan a no-downtime migration.

Project 4. AI Customer Support Platform

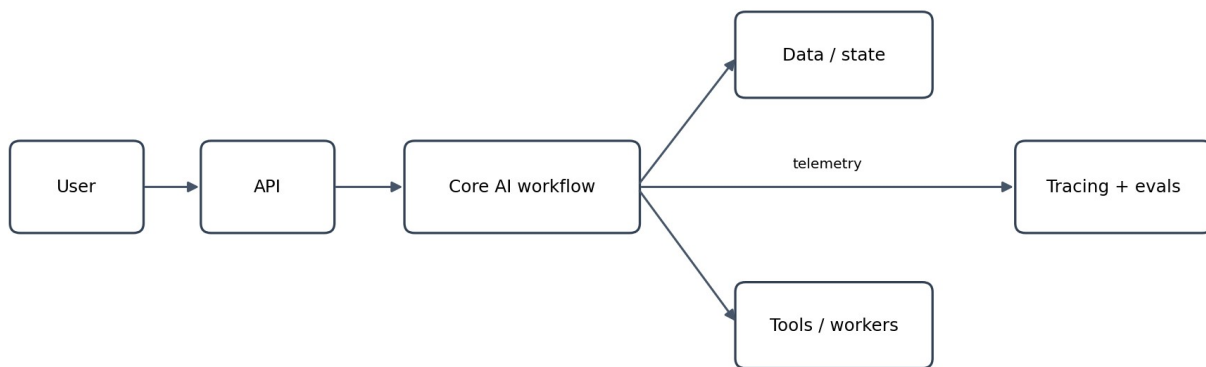
Objective

Build a production support assistant that routes simple questions, uses account tools safely and escalates uncertain cases to humans.

Suggested stack

API service, retrieval, account tools, Postgres, event queue, tracing, human-review dashboard.

AI Customer Support Platform - reference topology



AI Customer Support Platform - Milestones

1. FAQ/RAG baseline.
2. Typed intent/routing.
3. Read-only account tools.
4. Approval for sensitive side effects.
5. Human handoff package with evidence.
6. SLOs, degradation and per-tenant cost controls.

RULE: Do not add the next subsystem until you have a measurable baseline for the current one.

Evidence to collect

- Architecture diagram and data-flow diagram.
- Baseline load/eval results.
- A trace from one successful run and one failed run.
- A short decision record for every major technology choice.

AI Customer Support Platform - Failure Injection

Deliberately create the following conditions and document the observed behavior:

- **wrong account access** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **duplicate refund** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **stale policy** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **tool outage** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **unsafe auto-action** - record user-visible effect, telemetry, recovery path and whether the design needs to change.

Minimum reliability bar

- No silent data loss.
- No unbounded retry loop.
- No side effect without an idempotency or approval strategy.
- Every failure is visible in a trace or operational event.

AI Customer Support Platform - Evaluation

What this project teaches

Routing, guardrails, permissions, HITL, idempotent tools, operational reliability and product metrics.

Metrics

- Task success / user journey success.
- p50, p95 and timeout rate.
- Cost per successful task.
- Failure distribution by category.
- Human verification time for high-value outputs.

Release exercise

Create a candidate change, run offline regression tests, route 5% of synthetic traffic to it, and define an automatic rollback threshold before you inspect the results.

AI Customer Support Platform - Architecture Review

Review questions

- Where is the source of truth?
- Which components are derived and can be rebuilt?
- What is the concurrency/capacity bottleneck?
- Which operation is not safe to retry?
- What data must never cross tenant boundaries?
- What is the degraded mode?
- How will you migrate a major storage/model/index change?

Deliverables

- README with architecture and trade-offs.
- One-page runbook.
- Eval/load-test results.
- Failure-injection report.
- Recorded architecture walkthrough.

AI Customer Support Platform - Final Recall

EXPLAIN IT ALOUD: Describe the request path, state ownership, failure handling, observability and release strategy in under five minutes. If you cannot, simplify or redraw the architecture.

Stretch goals

- Introduce one multi-region or multi-tenant constraint.
- Add a cost ceiling and degradation strategy.
- Simulate a provider or database outage.
- Replace one core dependency and plan a no-downtime migration.

Project 5. AI SRE / Incident Response Agent

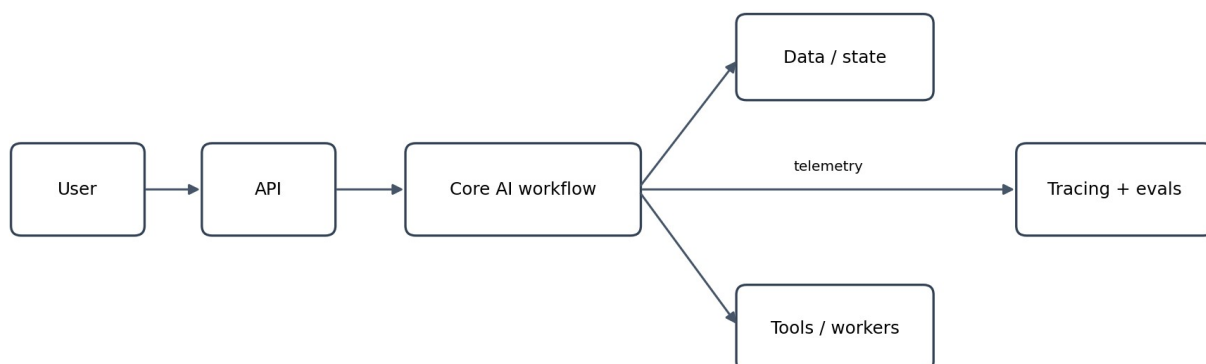
Objective

Build a read-heavy diagnostic agent that correlates metrics, logs, traces and deploys to produce evidence-backed incident hypotheses.

Suggested stack

OpenTelemetry, Prometheus-compatible metrics, log store, deployment metadata, runbooks, sandboxed tooling.

AI SRE / Incident Response Agent - reference topology



AI SRE / Incident Response Agent - Milestones

1. Read-only telemetry tools.
2. Incident timeline builder.
3. Hypothesis/evidence schema.
4. Runbook retrieval.
5. Human-approved reversible remediation.
6. Post-incident evals from real traces.

RULE: Do not add the next subsystem until you have a measurable baseline for the current one.

Evidence to collect

- Architecture diagram and data-flow diagram.
- Baseline load/eval results.
- A trace from one successful run and one failed run.
- A short decision record for every major technology choice.

AI SRE / Incident Response Agent - Failure Injection

Deliberately create the following conditions and document the observed behavior:

- **confirmation bias** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **wrong service scope** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **alert storm** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **unsafe remediation** - record user-visible effect, telemetry, recovery path and whether the design needs to change.
- **missing telemetry** - record user-visible effect, telemetry, recovery path and whether the design needs to change.

Minimum reliability bar

- No silent data loss.
- No unbounded retry loop.
- No side effect without an idempotency or approval strategy.
- Every failure is visible in a trace or operational event.

AI SRE / Incident Response Agent - Evaluation

What this project teaches

Observability, verification, tool policy, SLOs, human approval and production incident reasoning.

Metrics

- Task success / user journey success.
- p50, p95 and timeout rate.
- Cost per successful task.
- Failure distribution by category.
- Human verification time for high-value outputs.

Release exercise

Create a candidate change, run offline regression tests, route 5% of synthetic traffic to it, and define an automatic rollback threshold before you inspect the results.

AI SRE / Incident Response Agent - Architecture Review

Review questions

- Where is the source of truth?
- Which components are derived and can be rebuilt?
- What is the concurrency/capacity bottleneck?
- Which operation is not safe to retry?
- What data must never cross tenant boundaries?
- What is the degraded mode?
- How will you migrate a major storage/model/index change?

Deliverables

- README with architecture and trade-offs.
- One-page runbook.
- Eval/load-test results.
- Failure-injection report.
- Recorded architecture walkthrough.

AI SRE / Incident Response Agent - Final Recall

EXPLAIN IT ALOUD: Describe the request path, state ownership, failure handling, observability and release strategy in under five minutes. If you cannot, simplify or redraw the architecture.

Stretch goals

- Introduce one multi-region or multi-tenant constraint.
- Add a cost ceiling and degradation strategy.
- Simulate a provider or database outage.
- Replace one core dependency and plan a no-downtime migration.

APPENDICES

Appendix A - Equations Worth Remembering

Next-token prediction: $P(x_t | x_{<t})$

The model predicts the next token conditioned on prior tokens.

Softmax: $p_i = \exp(z_i) / \sum_j \exp(z_j)$

Converts logits to a probability distribution.

Scaled attention: $\text{softmax}(QK^T / \sqrt{d_k})V$

Weights value vectors by query-key compatibility.

Cosine similarity: $(a \cdot b) / (\|a\| \|b\|)$

Angle-based vector similarity used in semantic search.

Recall@k: relevant retrieved in top-k / total relevant

Measures how much needed evidence retrieval recovered.

Precision: $TP / (TP + FP)$

How many predicted positives are correct.

Recall: $TP / (TP + FN)$

How many actual positives are found.

Little's Law: $L = \lambda W$

Average work in system equals arrival rate times average time.

Error budget: $1 - SLO$

Allowed fraction of bad events.

Burn rate: observed error rate / allowed error rate

How quickly the error budget is being consumed.

Risk: $P(\text{failure}) \times \text{blast radius}$

Useful mental model for agent safety and release design.

Cost per success: total run cost / successful tasks

More meaningful than raw per-request spend.

Appendix B - Production Agent Release Checklist

Product

- ☐ User journey and success criteria defined
- ☐ Human fallback/escalation exists where needed
- ☐ Verification surface is appropriate to risk

Quality

- ☐ Core eval suite passes
- ☐ Regression cases pass
- ☐ Critical slices meet thresholds
- ☐ Judge calibrated against human labels

Runtime

- ☐ Run state is durable
- ☐ Deadlines and budgets enforced
- ☐ Retries bounded and classified
- ☐ Idempotency strategy exists for side effects

Security

- ☐ Least-privilege agent identity
- ☐ Sandbox/network policy tested
- ☐ Prompt-injection surfaces identified
- ☐ High-risk actions require approval

Operations

- ☐ Trace ID across model/tools/retrieval
- ☐ SLO and alerting defined
- ☐ Degraded modes tested
- ☐ Canary/rollback path tested

Economics

- ☐ Cost/run measured

- ☐ Cost/success measured
- ☐ Model routing justified by evals
- ☐ Context/tool-result waste reviewed

Appendix C - Source Index

Chip Huyen - Building LLM applications for production - <https://huyenchip.com/2023/04/11/llm-engineering.html>

Chip Huyen - Common pitfalls when building generative AI applications - <https://huyenchip.com/2025/01/16/ai-engineering-pitfalls.html>

Hugging Face LLM Course - <https://huggingface.co/learn/llm-course/chapter1/8>

Chip Huyen - Generation configurations - <https://huyenchip.com/2024/01/16/sampling.html>

Attention Is All You Need - <https://arxiv.org/abs/1706.03762>

The Annotated Transformer - <https://nlp.seas.harvard.edu/annotated-transformer/>

vLLM project - <https://github.com/vllm-project/vllm>

vLLM paper - PagedAttention - <https://arxiv.org/abs/2309.06180>

OpenAI - Structured Outputs - <https://developers.openai.com/api/docs/guides/structured-outputs>

Pydantic documentation - <https://docs.pydantic.dev/latest/>

Lilian Weng - Prompt Engineering - <https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/>

Anthropic prompt engineering docs - <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

Chip Huyen - Predictive Human Preference: From Model Ranking to Model Routing - <https://huyenchip.com/2024/02/28/predictive-human-preference.html>

OpenAI model guidance - <https://developers.openai.com/api/docs/guides/latest-model>

OpenAI embeddings guide - <https://developers.openai.com/api/docs/guides/embeddings>

Sentence Transformers documentation - <https://www.sbert.net/>

FAISS wiki - <https://github.com/facebookresearch/faiss/wiki/>

FAISS - Guidelines to choose an index - <https://github.com/facebookresearch/faiss/wiki/Guidelines-to-choose-an-index>

pgvector - <https://github.com/pgvector/pgvector>

Anthropic - Contextual Retrieval - <https://www.anthropic.com/engineering/contextual-retrieval>

LangChain text splitters concepts - https://python.langchain.com/docs/concepts/text_splitters/

Elasticsearch BM25 similarity - <https://www.elastic.co/guide/en/elasticsearch/reference/current/index-modules-similarity.html>

Sentence Transformers - Cross Encoders - https://www.sbert.net/examples/cross_encoder/applications/README.html

Eugene Yan - LLM patterns - <https://eugeneyan.com/writing/llm-patterns/>

BEIR benchmark paper - <https://arxiv.org/abs/2104.08663>

Information retrieval evaluation overview - Stanford IR book - <https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-of-ranked-retrieval-results-1.html>

Anthropic - Building effective agents - <https://www.anthropic.com/engineering/building-effective-agents>

OpenAI Agents SDK guide - <https://developers.openai.com/api/docs/guides/agents/sdk>

Anthropic - Writing effective tools for agents - <https://www.anthropic.com/engineering/writing-tools-for-agents>

ReAct paper - <https://arxiv.org/abs/2210.03629>

Anthropic - Effective context engineering for AI agents - <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Anthropic - Code execution with MCP - <https://www.anthropic.com/engineering/code-execution-with-mcp>

Anthropic - Effective harnesses for long-running agents - <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

Anthropic - How we built our multi-agent research system - <https://www.anthropic.com/engineering/multi-agent-research-system>

Anthropic - Building a C compiler with a team of parallel Claudes - <https://www.anthropic.com/engineering/building-c-compiler>

Hamel Husain - Putting Evals Into Production - <https://hamel.dev/notes/llm/ai-product-engineering/evals-production.html>

Anthropic - Demystifying evals for AI agents - <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

OpenAI - Agent evals - <https://developers.openai.com/api/docs/guides/agent-evals>

OpenTelemetry documentation - <https://opentelemetry.io/docs/>

OpenAI - Tracing - <https://developers.openai.com/api/docs/guides/agents-api/tracing>

OpenAI pricing - <https://openai.com/api/pricing/>

OpenAI - Latency optimization - <https://developers.openai.com/api/docs/guides/latency-optimization>

Google SRE Workbook - Alerting on SLOs - <https://sre.google/workbook/alerting-on-slos/>

AWS Builders Library - Timeouts, retries and backoff with jitter - <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>

Martin Fowler - Feature Toggles - <https://martinfowler.com/articles/feature-toggles.html>

Anthropic - How we contain Claude across products - <https://www.anthropic.com/engineering/how-we-contain-claude>

OWASP - LLM Prompt Injection - <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

OpenAI - Guardrails and approvals - <https://developers.openai.com/api/docs/guides/agents/guardrails-approvals>

Hamel Husain - AI Product Engineering notes - <https://hamel.dev/notes/llm/ai-product-engineering/>

Prompt Caching and Reuse - primary reference - <https://developers.openai.com/api/docs/guides/prompt-caching>

Model and Provider Failure Modes - primary reference - <https://developers.openai.com/api/docs/guides/rate-limits>

Background Jobs and Resumable Execution - primary reference - <https://developers.openai.com/api/docs/guides/background>

Production Checklists and Operational Readiness - primary reference -
<https://developers.openai.com/api/docs/guides/production-best-practices>

Appendix D - Glossary and Recall Questions

Agent: A model-driven runtime that can select actions/tools based on observations.

ANN: Approximate nearest-neighbor search.

Chunk: A retrieval unit derived from a larger source.

Context: Information visible to a model invocation.

Eval: A repeatable test of application behavior.

HNSW: Graph-based approximate-nearest-neighbor index.

Idempotency: Repeating an operation yields one effective business outcome.

KV cache: Stored attention keys/values from prior tokens during decoding.

RAG: Retrieval-augmented generation.

Reranker: Model that rescales a candidate set for relevance.

Run state: Durable machine state required to resume a workflow.

SLI/SLO: Measured service indicator and its reliability target.

Span: One timed operation inside a distributed trace.

Tool: Typed capability exposed to an agent.

Trajectory: Sequence of actions and observations in an agent run.

Master recall questions

1. When should I not use an LLM?
2. Where does truth live?
3. What context should the model see right now?
4. How do I know retrieval found the required evidence?
5. What prevents one bad agent run from causing broad damage?
6. What happens after a timeout: did the side effect occur?
7. How will this run resume after a process crash?
8. What metric would reveal a regression hidden by the average?
9. How do we degrade under overload rather than collapse?
10. What evidence lets a human verify the output quickly?